claude-windows / ed7a77e2-08ae-4342-aa1f-da67fd166031

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ed7a77e2-08ae-4342-aa1f-da67fd166031.jsonl`
- SHA-256: `558d99b062b84296730f757190ae3b17a8b3c48a2ed3d988fcc673f310c73a16`
- Source modified: `2026-06-20T12:47:58+00:00`
- Imported at: `2026-07-05T16:48:19+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `ed7a77e2-08ae-4342-aa1f-da67fd166031`

Transcript

1. user (2026-06-20T09:38:00.873Z)

In badminton-highlight-indexer, implement the Google Drive storage backend so `gdrive://` works as a first-class worker input (the owner wants GDrive as an upload option alongside bucket URLs and local upload). Currently `backend/storage/gdrive.py::GDriveStorage` is a stub that raises NotImplementedError, and the only Drive path is `gdown <file-id>` ? local.

Scope:
- Implement `GDriveStorage` (fetch_to_local / put / exists / stat / list / delete) against `pydrive2` behind the existing `storage-gdrive` extra (see `pyproject.toml`), with a file-id URI model (`gdrive://<file-id>` or folder-id for list). Auth via a service-account JSON (mirror the GCS pattern: creds from env/ADC, never config).
- Wire it through `backend/storage/__init__.py::_backend_class` (already routes "gdrive") and `parse_uri`.
- Add unit tests mocking the Drive client (no network), mirroring tests/test_*storage patterns.
- Update `deploy/gcp/README.md` §5 (Upload options) + `docs/SETUP_NEW_MACHINE.md` §7 to mark gdrive:// as implemented.
- Keep the import lazy (cloud SDK only when a gdrive:// URI is used). Run `python run_all_tests.py` + ruff + mypy green before opening a PR off origin/master.

Context: the worker (`python -m backend.worker infer --video-uri ...`) resolves any URI via the storage layer; bucket (gcs://, s3://) and local already work. This closes the Drive gap noted in ADR-010/the GCP runbook.

2. assistant (2026-06-20T09:38:07.363Z)

I'll start by following the repo-freshness and branching conventions, then read the relevant code. Let me kick off some parallel exploration.

3. user (2026-06-20T09:38:12.380Z)

```
---BRANCH---
docs/gcp-pivot-runbook
---STATUS---
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
---LOG vs origin/master---
e2e9754 docs(gcp): add Spot-T4 fast path (likely skips the GPUS_ALL_REGIONS wait)
0ebfd2e docs(decoupled): GCP pivot runbook + DO?GCP ADR-010 + handoff/setup updates
```

4. user (2026-06-20T09:38:12.419Z)

```
1       """Google Drive backend ? placeholder for a focused follow-up PR.
2
3       Drive is the owner's initial-testing store (2 TB via Google AI Plus), but it is a
genuinely
4       *second-class* backend: a file-id model (no real path namespace) and no presigned URLs,
```

```
plus
5       OAuth (``drive.file``) vs service-account auth choices that need the real Drive to
validate.
6       It is therefore implemented in a dedicated follow-up (via ``pydrive2``, the ``storage-
gdrive``
7       extra) rather than guessed at here ? keeping the first-class S3/GCS path clean and
tested.
8
9       Recognising the ``gdrive://`` scheme now (with a clear error) beats a confusing
10      ``unknown backend`` later.
11      """
12      from __future__ import annotations
13
14      from backend.storage.base import StorageBackend
15
16
17      class GDriveStorage(StorageBackend):
18          def __init__(self, **_ignored) -> None:
19              raise NotImplementedError(
20                  "The Google Drive backend (gdrive://) is not implemented yet ? it lands in a
"
21                  "follow-up PR (pydrive2 + the 'storage-gdrive' extra), validated against the
"
22                  "owner's Drive. Use local://, s3:// or gcs:// for now."
23              )
24
25          def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:  # pragma: no
cover
26              raise NotImplementedError
27
28          def put(self, local_path, ref, *, content_type=None):  # pragma: no cover
29              raise NotImplementedError
30
31          def exists(self, ref) -> bool:  # pragma: no cover
32              raise NotImplementedError
33
34          def stat(self, ref):  # pragma: no cover
35              raise NotImplementedError
36
37          def list(self, prefix):  # pragma: no cover
38              raise NotImplementedError
39
40          def delete(self, ref) -> None:  # pragma: no cover
41              raise NotImplementedError
42
```

5. user (2026-06-20T09:38:12.432Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Drive
(follow-up).
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local URI is actually used (install the ``storage-s3`` /
``storage-gcs``
5       / ``storage-gdrive`` extras). The default ``local`` backend needs no third-party deps
and
6       preserves today's behaviour byte-for-byte.
7
8       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
9       ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
10      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
11      """
12      from __future__ import annotations
```

```

    import os
    from typing import Any, Dict, Iterator, Optional

    from backend.storage.base import StorageBackend
    from backend.storage.ref import StorageRef, StorageStat, parse_uri

    __all__ = [
        "StorageBackend", "StorageRef", "StorageStat", "parse_uri",
        "get_storage", "fetch", "put", "list_uris",
    ]


    def _backend_class(name: str):
        """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
        if name == "local":
            from backend.storage.local import LocalStorage
            return LocalStorage
        if name == "s3":
            from backend.storage.s3 import S3Storage
            return S3Storage
        if name == "gcs":
            from backend.storage.gcs import GCSStorage
            return GCSStorage
        if name == "gdrive":
            from backend.storage.gdrive import GDriveStorage
            return GDriveStorage
        raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")


    def get_storage(backend: Optional[str] = None, *,
                    config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
        """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
        the implementation; its per-backend settings come from ``config[backend]`` merged
with
        ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
        cfg = config or {}
        name = backend or cfg.get("backend", "local")
        settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
        settings.update(overrides)
        return _backend_class(name)(**settings)


    def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
        """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
        (no copy); for s3/gcs it downloads to ``dst``."""
        ref = parse_uri(uri)
        return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)


    def put(local_path: "str | os.PathLike[str]", uri: str, *,
            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
        """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
        ref = parse_uri(uri)
        get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
        return uri

```

```
70
71      def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
72          """Yield the URI of every object under ``prefix_uri``."""
73          ref = parse_uri(prefix_uri)
74          for r in get_storage(ref.backend, config=config).list(ref):
75              yield r.uri
76
```

6. assistant (2026-06-20T09:38:15.417Z)

Now let me read the base class, ref parsing, the GCS backend (to mirror), and the existing storage tests.

7. user (2026-06-20T09:38:16.372Z)

```
1       """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3       Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). Concrete backends
4       live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5       pulls ``boto3`` / ``google-cloud-storage`` / ``pydrive2``. See
``PLATFORM_ARCHITECTURE.md``
6       §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7       """
8       from __future__ import annotations
9
10      import os
11      from abc import ABC, abstractmethod
12      from typing import Iterator
13
14      from backend.storage.ref import StorageRef, StorageStat
15
16
17      class StorageBackend(ABC):
18          """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19          worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21          @abstractmethod
22          def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                             *, overwrite: bool = False) -> str:
24              """Make the object available as a local file; return its local path. For the
25              ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27          @abstractmethod
28          def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29                  content_type: "str | None" = None) -> StorageRef:
30              """Upload/copy a local file to ``ref``; return ``ref``."""
31
32          @abstractmethod
33          def exists(self, ref: StorageRef) -> bool: ...
34
35          @abstractmethod
36          def stat(self, ref: StorageRef) -> StorageStat: ...
37
38          @abstractmethod
39          def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40              """Yield a ``StorageRef`` per object under ``prefix``."""
41
42          @abstractmethod
43          def delete(self, ref: StorageRef) -> None: ...
44
45          def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
```

```
str:
46              """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47              sign (``local`` cannot; Drive only emulates) ? callers must treat
48              ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49              raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50
```

8. user (2026-06-20T09:38:16.844Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<folderId>/<path>  -> gdrive   (follow-up backend)
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket or the Drive folder-id;
37          for ``local`` it is empty and ``key`` carries the filesystem path."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
```

```
        """
        s = str(uri)
        m = _SCHEME_RE.match(s)
        if not m:
            return StorageRef("local", "", s, f"local://{s}")
        scheme = m.group(1).lower()
        rest = s[m.end():]
        if scheme == "gs":
            scheme = "gcs"
        if scheme == "local":
            return StorageRef("local", "", rest, s)
        if scheme in _KNOWN:
            bucket, _, key = rest.partition("/")
            return StorageRef(scheme, bucket, key, s)
        raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")

```

9. user (2026-06-20T09:38:17.130Z)

```
    """Google Cloud Storage backend ? first-class object buckets.

    Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
    install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
    (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
    never from config. A ``client`` may be injected for tests.
    """
    from __future__ import annotations

    from datetime import timedelta
    from typing import Any, Iterator, Optional

    from backend.storage.base import StorageBackend
    from backend.storage.ref import StorageRef, StorageStat


    class GCSStorage(StorageBackend):
        def __init__(self, *, client: Any = None, project: Optional[str] = None,
                     signed_url_ttl: int = 3600, **_ignored) -> None:
            self._ttl = signed_url_ttl
            if client is not None:
                self._client = client
                return
            from google.cloud import storage  # type: ignore  # lazy: only when a gcs:// URI
is used
            self._client = storage.Client(project=project)

        def _blob(self, ref: StorageRef):
            return self._client.bucket(ref.bucket).blob(ref.key)

        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("GCSStorage.fetch_to_local requires a dst path")
            import os
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._blob(ref).download_to_filename(dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
```

```
43              return ref
44
45          def exists(self, ref) -> bool:
46              return bool(self._blob(ref).exists())
47
48          def stat(self, ref) -> StorageStat:
49              b = self._blob(ref)
50              b.reload()
51              return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                                 content_type=b.content_type)
53
54          def list(self, prefix) -> Iterator[StorageRef]:
55              for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56                  yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
57
58          def delete(self, ref) -> None:
59              self._blob(ref).delete()
60
61          def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62              return self._blob(ref).generate_signed_url(
63                  version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64
```

10. assistant (2026-06-20T09:38:20.410Z)

Now the S3 backend, local backend, the existing storage tests, and the pyproject extras.

11. user (2026-06-20T09:38:21.363Z)

```
1       """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3       One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4       only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5       ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7       Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8       style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9       / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10      (tests pass a fake; production lets the constructor build one via boto3).
11      """
12      from __future__ import annotations
13
14      import os
15      from typing import Any, Iterator, Optional
16
17      from backend.storage.base import StorageBackend
18      from backend.storage.ref import StorageRef, StorageStat
19
20
21      class S3Storage(StorageBackend):
22          def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                       region: Optional[str] = None, addressing_style: str = "auto",
24                       signed_url_ttl: int = 3600, **_ignored) -> None:
25              self._ttl = signed_url_ttl
26              if client is not None:
27                  self._client = client
28                  return
29              import boto3  # type: ignore  # lazy: only when an s3:// URI is actually used
30              from botocore.config import Config  # type: ignore
31              self._client = boto3.client(
32                  "s3",
```

```
                endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
                region_name=region,
                config=Config(s3={"addressing_style": addressing_style}),
            )

        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            if dst is None:
                raise ValueError("S3Storage.fetch_to_local requires a dst path")
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            self._client.download_file(ref.bucket, ref.key, dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            extra = {"ContentType": content_type} if content_type else None
            self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
            return ref

        def exists(self, ref) -> bool:
            try:
                self._client.head_object(Bucket=ref.bucket, Key=ref.key)
                return True
            except Exception:  # noqa: BLE001 - any boto/client error means "not found /
unreachable"
                return False

        def stat(self, ref) -> StorageStat:
            h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
            return StorageStat(size=int(h.get("ContentLength", 0)),
modified=h.get("LastModified"),
                               etag=h.get("ETag"), content_type=h.get("ContentType"))

        def list(self, prefix) -> Iterator[StorageRef]:
            paginator = self._client.get_paginator("list_objects_v2")
            for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
                for obj in page.get("Contents", []) or []:
                    key = obj["Key"]
                    yield StorageRef("s3", prefix.bucket, key,
f"s3://{prefix.bucket}/{key}")

        def delete(self, ref) -> None:
            self._client.delete_object(Bucket=ref.bucket, Key=ref.key)

        def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
            op = "get_object" if method.upper() == "GET" else "put_object"
            return self._client.generate_presigned_url(
                op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or
self._ttl)

```

12. user (2026-06-20T09:38:21.841Z)

```
    """Local-filesystem storage backend ? the default, zero new dependencies.

    Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
    file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
    the whole pipeline behaves exactly as before the storage layer existed.
    """
    from __future__ import annotations

    import os
```

```
    import shutil
    from datetime import datetime
    from typing import Iterator

    from backend.storage.base import StorageBackend
    from backend.storage.ref import StorageRef, StorageStat


    class LocalStorage(StorageBackend):
        def __init__(self, *, base_dir: str = "", **_ignored) -> None:
            # Relative keys resolve under base_dir when set; absolute keys are used as-is.
            self.base_dir = base_dir

        def _path(self, ref: StorageRef) -> str:
            p = ref.key
            if self.base_dir and not os.path.isabs(p):
                return os.path.join(self.base_dir, p)
            return p

        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
            # The object is already on the local FS ? return its path (identity). ``dst`` is
            # honoured only if it names a *different* location (a copy is then made).
            src = self._path(ref)
            if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
                return src
            dst = str(dst)
            if os.path.exists(dst) and not overwrite:
                return dst
            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
            shutil.copyfile(src, dst)
            return dst

        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
            dst = self._path(ref)
            if os.path.abspath(str(local_path)) != os.path.abspath(dst):
                os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
                shutil.copyfile(str(local_path), dst)
            return ref

        def exists(self, ref) -> bool:
            return os.path.exists(self._path(ref))

        def stat(self, ref) -> StorageStat:
            st = os.stat(self._path(ref))
            return StorageStat(size=st.st_size,
modified=datetime.fromtimestamp(st.st_mtime))

        def list(self, prefix) -> Iterator[StorageRef]:
            base = self._path(prefix)
            if os.path.isdir(base):
                for root, _dirs, files in os.walk(base):
                    for fn in sorted(files):
                        full = os.path.join(root, fn)
                        yield StorageRef("local", "", full, f"local://{full}")
            elif os.path.exists(base):
                yield StorageRef("local", "", base, f"local://{base}")

        def delete(self, ref) -> None:
            path = self._path(ref)
            if os.path.exists(path):
                os.remove(path)

```

13. user (2026-06-20T09:38:22.235Z)

tests\test_storage.py

14. assistant (2026-06-20T09:38:24.820Z)

Now let me read the existing storage tests and the pyproject extras to understand the patterns I need to mirror.

15. user (2026-06-20T09:38:25.784Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets.
2
3       Cloud backends are exercised with injected FAKE clients, so these tests need neither
4       boto3/google-cloud-storage installed nor any network ? matching the repo's offline
ethos.
5       """
6       import subprocess
7       import sys
8
9       import pytest
10
11      from backend import storage
12      from backend.storage import StorageBackend, fetch, get_storage, list_uris, parse_uri,
put
13      from backend.storage.local import LocalStorage
14      from backend.storage.s3 import S3Storage
15      from backend.storage.gcs import GCSStorage
16
17
18      # ---------------------------------------------------------------- URI parsing
19
20      @pytest.mark.parametrize("uri,backend,bucket,key", [
21          ("/data/x.mp4", "local", "", "/data/x.mp4"),
22          ("rel/x.mp4", "local", "", "rel/x.mp4"),
23          ("C:/Users/a/x.mp4", "local", "", "C:/Users/a/x.mp4"),       # Windows path: no
':://'
24          ("C:\\Users\\a\\x.mp4", "local", "", "C:\\Users\\a\\x.mp4"),
25          ("local:///abs/x.mp4", "local", "", "/abs/x.mp4"),
26          ("s3://rally/videos/m.mp4", "s3", "rally", "videos/m.mp4"),
27          ("gs://rally-eu/w/model.json", "gcs", "rally-eu", "w/model.json"),   # gs:// -> gcs
28          ("gcs://rally/x", "gcs", "rally", "x"),
29          ("gdrive://FOLDERID/out/reel.mp4", "gdrive", "FOLDERID", "out/reel.mp4"),
30      ])
31      def test_parse_uri(uri, backend, bucket, key):
32          ref = parse_uri(uri)
33          assert (ref.backend, ref.bucket, ref.key) == (backend, bucket, key)
34
35
36      def test_parse_uri_rejects_unknown_scheme():
37          with pytest.raises(ValueError):
38              parse_uri("ftp://host/x")
39
40
41      # ---------------------------------------------------------------- local backend
42
43      def test_local_fetch_is_identity_passthrough(tmp_path):
44          src = tmp_path / "v.mp4"
45          src.write_bytes(b"abc")
46          # No dst => returns the source path unchanged (no copy): preserves today's
behaviour.
47          assert fetch(str(src)) == str(src)
48
49
50      def test_local_put_and_roundtrip(tmp_path):
51          src = tmp_path / "in.csv"
52          src.write_text("Frame,Visibility,X,Y\n")
```

```
        dst_uri = str(tmp_path / "out" / "copy.csv")
        put(str(src), dst_uri)
        assert (tmp_path / "out" / "copy.csv").read_text() == src.read_text()


    def test_local_exists_stat_list_delete(tmp_path):
        be = LocalStorage()
        (tmp_path / "a.txt").write_text("hello")
        (tmp_path / "sub").mkdir()
        (tmp_path / "sub" / "b.txt").write_text("hi")
        assert be.exists(parse_uri(str(tmp_path / "a.txt")))
        assert not be.exists(parse_uri(str(tmp_path / "nope.txt")))
        assert be.stat(parse_uri(str(tmp_path / "a.txt"))).size == 5
        listed = sorted(r.key for r in be.list(parse_uri(str(tmp_path))))
        assert listed == sorted([str(tmp_path / "a.txt"), str(tmp_path / "sub" / "b.txt")])
        be.delete(parse_uri(str(tmp_path / "a.txt")))
        assert not (tmp_path / "a.txt").exists()


    def test_local_signed_url_raises():
        with pytest.raises(NotImplementedError):
            LocalStorage().signed_url(parse_uri("/x"))


    def test_list_uris_helper(tmp_path):
        (tmp_path / "x.txt").write_text("1")
        (tmp_path / "y.txt").write_text("2")
        uris = sorted(list_uris(str(tmp_path)))
        assert all(u.startswith("local://") for u in uris)
        assert len(uris) == 2


    # ---------------------------------------------------------------- registry / get_storage

    def test_get_storage_defaults_to_local():
        assert isinstance(get_storage(), LocalStorage)
        assert isinstance(get_storage(config={"backend": "local"}), LocalStorage)


    def test_get_storage_unknown_backend_raises():
        with pytest.raises(ValueError):
            get_storage("azure")


    def test_gdrive_is_recognised_but_not_implemented():
        # The scheme parses (so callers get a clear message), construction is a clean
NotImplemented.
        assert parse_uri("gdrive://f/x").backend == "gdrive"
        with pytest.raises(NotImplementedError):
            get_storage("gdrive")


    # ---------------------------------------------------------------- S3 (fake client)

    class _FakeS3Client:
        def __init__(self):
            self.store = {}

        def upload_file(self, src, bucket, key, ExtraArgs=None):
            with open(src, "rb") as f:
                self.store[(bucket, key)] = f.read()

        def download_file(self, bucket, key, dst):
            with open(dst, "wb") as f:
                f.write(self.store[(bucket, key)])
```

```

        def head_object(self, Bucket, Key):
            if (Bucket, Key) not in self.store:
                raise KeyError("404")
            return {"ContentLength": len(self.store[(Bucket, Key)]), "ETag": '"e"',
                    "ContentType": "video/mp4"}

        def get_paginator(self, _op):
            store = self.store

            class _P:
                def paginate(self, Bucket, Prefix=""):
                    yield {"Contents": [{"Key": k} for (b, k) in store
                                        if b == Bucket and k.startswith(Prefix)]}
            return _P()

        def delete_object(self, Bucket, Key):
            self.store.pop((Bucket, Key), None)

        def generate_presigned_url(self, op, Params, ExpiresIn):
            return f"https://s3.example/{Params['Bucket']}/{Params['Key']}?X-Amz-
Expires={ExpiresIn}"


    def test_s3_backend_roundtrip_with_fake_client(tmp_path):
        be = S3Storage(client=_FakeS3Client())
        assert isinstance(be, StorageBackend)
        src = tmp_path / "m.mp4"
        src.write_bytes(b"VIDEO")
        ref = parse_uri("s3://rally/videos/m.mp4")
        be.put(str(src), ref, content_type="video/mp4")
        assert be.exists(ref)
        assert be.stat(ref).size == 5
        dst = tmp_path / "dl" / "m.mp4"
        assert be.fetch_to_local(ref, str(dst)) == str(dst)
        assert dst.read_bytes() == b"VIDEO"
        assert [r.uri for r in be.list(parse_uri("s3://rally/videos/"))] ==
["s3://rally/videos/m.mp4"]
        url = be.signed_url(ref, ttl=900)
        assert url.startswith("https://s3.example/rally/videos/m.mp4") and "900" in url
        be.delete(ref)
        assert not be.exists(ref)


    def test_s3_exists_false_on_missing():
        assert S3Storage(client=_FakeS3Client()).exists(parse_uri("s3://b/none")) is False


    # ---------------------------------------------------------------- GCS (fake client)

    class _FakeBlob:
        def __init__(self, store, bucket, name):
            self.store, self.bucket, self.name = store, bucket, name
            self.size = self.updated = self.etag = self.content_type = None

        def upload_from_filename(self, src, content_type=None):
            with open(src, "rb") as f:
                self.store[(self.bucket, self.name)] = f.read()
            self.content_type = content_type

        def download_to_filename(self, dst):
            with open(dst, "wb") as f:
                f.write(self.store[(self.bucket, self.name)])

        def exists(self):
```

```
            return (self.bucket, self.name) in self.store

        def reload(self):
            self.size = len(self.store[(self.bucket, self.name)])

        def delete(self):
            self.store.pop((self.bucket, self.name), None)

        def generate_signed_url(self, version, expiration, method):
            return f"https://gcs.example/{self.bucket}/{self.name}?version={version}"


    class _FakeGCSClient:
        def __init__(self):
            self.store = {}

        def bucket(self, name):
            return type("_B", (), {"blob": lambda _self, key: _FakeBlob(self.store, name,
key)})()

        def list_blobs(self, bucket, prefix=""):
            return [_FakeBlob(self.store, bucket, k) for (b, k) in self.store
                    if b == bucket and k.startswith(prefix)]


    def test_gcs_backend_roundtrip_with_fake_client(tmp_path):
        be = GCSStorage(client=_FakeGCSClient())
        src = tmp_path / "w.json"
        src.write_text("{}")
        ref = parse_uri("gcs://rally/weights/w.json")
        be.put(str(src), ref)
        assert be.exists(ref)
        assert be.stat(ref).size == 2
        dst = tmp_path / "dl" / "w.json"
        be.fetch_to_local(ref, str(dst))
        assert dst.read_text() == "{}"
        assert [r.uri for r in be.list(parse_uri("gcs://rally/weights/"))] ==
["gcs://rally/weights/w.json"]
        assert be.signed_url(ref).startswith("https://gcs.example/rally/weights/w.json")
        be.delete(ref)
        assert not be.exists(ref)


    # ------------------------------------------------------------- lazy-import hygiene

    def test_importing_storage_does_not_pull_cloud_sdks():
        # `import backend.storage` must not eagerly import boto3 / google-cloud-storage, so
the
        # default install (and the CI lanes without those extras) stays clean.
        code = (
            "import sys, backend.storage; "
            "assert 'boto3' not in sys.modules, 'boto3 imported eagerly'; "
            "assert 'google.cloud.storage' not in sys.modules, 'gcs imported eagerly'; "
            "print('clean')"
        )
        r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True)
        assert r.returncode == 0, r.stderr
        assert "clean" in r.stdout


    def test_config_storage_defaults_are_non_breaking():
        from backend.config import StorageConfig
        sc = StorageConfig()
        assert sc.backend == "local" and sc.output_root == ""
        assert isinstance(storage.get_storage(config=sc.model_dump()), LocalStorage)
```

16. user (2026-06-20T09:38:25.898Z)

```
1       # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2       #
3       # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4       # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5       # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6       # auto-detection would otherwise miss the un-__init__'d `backend` root.
7       #
8       # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9       # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10      # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11      # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12      [build-system]
13      requires = ["hatchling"]
14      build-backend = "hatchling.build"
15
16      [project]
17      name = "badminton-highlight-indexer"
18      version = "0.1.0"
19      description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20      readme = "README.md"
21      requires-python = ">=3.10"
22      license = { text = "MIT" }
23      authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24      keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26      # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27      dependencies = [
28          "fastapi>=0.111.0",
29          "uvicorn>=0.30.1",
30          "opencv-python>=4.9.0.80",
31          "numpy>=1.26.4",
32          "pydantic>=2.7.1",
33          "jinja2>=3.1.4",
34          "python-dotenv>=1.0.0",
35          "google-genai>=2.4.0",
36          "supervision>=0.21.0,<0.30.0",
37      ]
38
39      [project.optional-dependencies]
40      # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41      dev = [
42          "pytest",
43          "pytest-cov",
44          "httpx",        # fastapi TestClient transport
45          "ruff",
46          "mypy",
47      ]
48
49      # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50      # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51      # fetched over SSH; if you lack access, simply omit this group ? reporting
52      # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53      #   pip install -e ../sports-obsreport --no-build-isolation
54      reporting = [
55          "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56      ]
57
58      # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59      # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60      # express, so they are NOT declarable here. Install them out-of-band instead:
```

```
61      #   # CUDA 12.4 box:
62      #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63      #   # CPU-only box (also what CI uses):
64      #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65      gpu = []
66
67      # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68      # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69      # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70      # DO Spaces + MinIO via endpoint_url.)
71      storage-s3 = ["boto3>=1.34"]
72      storage-gcs = ["google-cloud-storage>=2.16"]
73      storage-gdrive = ["pydrive2>=1.19"]   # follow-up backend (gdrive:// not implemented
yet)
74
75      [project.scripts]
76      # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77      indexer = "backend.main:main"
78
79      [tool.hatch.metadata]
80      # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81      # reference*); hatchling requires opting in to allow those in metadata.
82      allow-direct-references = true
83
84      [tool.hatch.build.targets.wheel]
85      # `backend` is a namespace package (no __init__.py); list both import roots
86      # explicitly so an editable/wheel install exposes `import backend` and
87      # `import training` regardless of cwd.
88      packages = ["backend", "training"]
89
```

17. assistant (2026-06-20T09:40:24.258Z)

Let me read the docs to update and find all references to the gdrive stub / gdown path.

18. user (2026-06-20T09:40:26.968Z)

```
=== gdrive refs ===
backend/config/models.py:294:    + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage;
``gdrive`` = Google Drive
backend/config/models.py:298:    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
backend/config/models.py:304:    gdrive: dict[str, Any] = Field(default_factory=dict)
backend/storage/__init__.py:5:/ ``storage-gdrive`` extras). The default ``local`` backend needs
no third-party deps and
backend/storage/__init__.py:37:    if name == "gdrive":
backend/storage/__init__.py:38:        from backend.storage.gdrive import GDriveStorage
backend/storage/__init__.py:39:        return GDriveStorage
backend/storage/__init__.py:40:    raise ValueError(f"unknown storage backend: {name!r}
(expected local|s3|gcs|gdrive)")
backend/storage/base.py:5:pulls ``boto3`` / ``google-cloud-storage`` / ``pydrive2``. See
``PLATFORM_ARCHITECTURE.md``
backend/storage/gdrive.py:6:It is therefore implemented in a dedicated follow-up (via
``pydrive2``, the ``storage-gdrive``
backend/storage/gdrive.py:9:Recognising the ``gdrive://`` scheme now (with a clear error) beats
a confusing
backend/storage/gdrive.py:17:class GDriveStorage(StorageBackend):
backend/storage/gdrive.py:20:            "The Google Drive backend (gdrive://) is not
implemented yet ? it lands in a "
backend/storage/gdrive.py:21:            "follow-up PR (pydrive2 + the 'storage-gdrive' extra),
validated against the "
backend/storage/ref.py:9:    gdrive://<folderId>/<path>  -> gdrive   (follow-up backend)
backend/storage/ref.py:22:_KNOWN = ("local", "s3", "gcs", "gdrive")
deploy/gcp/README.md:172:- **Google Drive:** `gdrive://` is not yet a first-class backend
```

(raises a clear error). Use
deploy/gcp/README.md:173: `gdown <file-id> -O clip.mp4` to pull to local, then pass the local
path. *(A first-class `gdrive://`
docs/DECOUPLED_COMPUTE/HANDOFF.md:22:- **Upload/run options** for inference: GCS bucket URI
(recommended), local path/scp, or GDrive via
docs/DECOUPLED_COMPUTE/HANDOFF.md:23: `gdown`?local (`gdrive://` backend is a tracked follow-
up). See `deploy/gcp/README.md` §5.
docs/DECOUPLED_COMPUTE/HANDOFF.md:41:| #249 | Storage layer ? `local` + **S3 (AWS/DO
Spaces/R2/MinIO)** + GCS, URI-driven; gdrive stub | `backend/storage/` |
docs/DECOUPLED_COMPUTE/HANDOFF.md:60:- **Drive corpus** (source): folder
`1sHGL7iacnmM80ChT-p2cKy638gK504T_` (gdown-able by file-id). 7 `[Done]` = video+labels;
docs/PLATFORM_ARCHITECTURE.md:125: local / S3 (`s3fs`) / GCS (`gcsfs`) / Azure (`adlfs`) +
Google Drive via `gdrivefs`,
docs/PLATFORM_ARCHITECTURE.md:129:- **Backends:** `local` (upload dir), `gdrive`, `s3`, `gcs`,
`azure`. **BYO-storage**:
docs/PLATFORM_ARCHITECTURE.md:477:| **P2** | **`StorageBackend` registry + BYO-storage** | new
ABC+registry; `local`?`gdrive`/`s3`/`gcs`; replace `video_path` with `StorageRef`; encrypted
per-tenant creds |
docs/PLATFORM_ARCHITECTURE.md:564:**Storage abstraction:** [fsspec](https://fsspec.com/) ·
[cloudpathlib](https://github.com/drivendataorg/cloudpathlib) ·
[smart_open](https://github.com/piskvorky/smart_open) · [Apache
Libcloud](https://libcloud.apache.org/) · [gdrive-fsspec](https://github.com/fsspec/gdrive-
fsspec) · [dropboxdrivefs](https://github.com/fsspec/dropboxdrivefs) · [AWS S3 presigned
URLs](https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-presigned-url.html) · [token-
based S3 with STS](https://oneuptime.com/blog/post/2026-02-12-token-based-access-s3-sts/view) ·
[signed URLs across clouds](https://medium.com/@shaikhsarwan49/signed-urls-in-aws-gcp-and-azure-
a-beginner-friendly-guide-e2e549425865)
docs/SETUP_NEW_MACHINE.md:214:`gdrive://` is recognised; the backend is a focused follow-up (for
the read-only golden corpus).
tests/test_storage.py:29:   ("gdrive://FOLDERID/out/reel.mp4", "gdrive", "FOLDERID",
"out/reel.mp4"),
tests/test_storage.py:97:def test_gdrive_is_recognised_but_not_implemented():
tests/test_storage.py:99:   assert parse_uri("gdrive://f/x").backend == "gdrive"
tests/test_storage.py:101:   get_storage("gdrive")

19. user (2026-06-20T09:40:27.104Z)

1   # GCP provisioning runbook ? DECOUPLED_COMPUTE GPU + storage (asia-south1)
2
3   Co-located **GPU VM + GCS bucket in ONE region** (asia-south1 / Mumbai) for the first
real
4   (non-mock) WASB training + inference. This is the **DO?GCP pivot** (ADR-010):
DigitalOcean GPU
5   Droplets are North-America-only (no Singapore/India GPU), so we moved GPU + storage to
GCP, which
6   has GPUs in asia-south1 (Mumbai) and asia-southeast1 (Singapore). Owner picked
Mumbai (closest
7   to Bangalore ? lowest latency).
8
9   > **Cost discipline (read first):** a running GPU VM is the only meaningful cost. Use
the cheapest
10   > adequate GPU (**T4**), **STOP or DELETE the VM the moment a run finishes** (a stopped
VM bills only
11   > for its disk), and keep the **$100 budget alert** live. `teardown.sh` switches
everything off.
12
13   ---
14
15   ## 0. Live resources (created 2026-06-20 ? the "switch-off" inventory)
16
17 | Resource | Identifier | Cost when idle | How to switch off |
18 |---|---|---|---|
19 | Project | `gen-lang-client-0306026826` (billing **Khelsutra Billing**
`01420D-419EE0-53D83C`) | ? | n/a (shared Gemini project) |
20 | GCS bucket | `gs://khelsutra-rally-corpus` (asia-south1, UBLA) | ~$0.02/GB/mo storage

```
| `gcloud storage rm -r` (? data loss) |
21      | Service account | `rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`
(`roles/storage.objectAdmin`) | $0 | `gcloud iam service-accounts delete` |
22      | SA key | `~/.gcp/rally-worker-sa.json` (local only, **never** committed) | $0 | delete
the file + the key |
23      | Budget alert | `rally-100usd-guard` ? $100, alerts at 50/90/100% | $0 | `gcloud
billing budgets delete` |
24      | **GPU VM** | **NOT created** ? blocked on `GPUS_ALL_REGIONS` quota (see §2) |
**~$0.35/hr (T4) running, ~$0.01/hr stopped** | `teardown.sh` (stop or delete) |
25      | APIs enabled | serviceusage, compute, cloudbilling, billingbudgets, iam,
cloudresourcemanager | $0 | leave on (free) |
26
27      Run `bash deploy/gcp/teardown.sh` (see §6) to stop/delete the billable bits between
sessions.
28
29      ---
30
31      ## 1. One-time project bootstrap (owner, console ? already done 2026-06-20)
32
33      A bare Gemini/AI-Studio project has the **Service Usage API disabled**, which blocks
*all* gcloud
34      provisioning (gcloud's own `services enable` needs that API ? chicken-and-egg). These
were enabled
35      in the console; recorded here so a fresh project can be bootstrapped the same way:
36
37      1. **Service Usage API** ?
https://console.developers.google.com/apis/api/serviceusage.googleapis.com/overview?project=gen-
lang-client-0306026826
38      2. **Link a billing account** (a payment method) ?
https://console.cloud.google.com/billing/linkedaccount?project=gen-lang-client-0306026826
39      3. After (1), the rest enable via gcloud (done): `gcloud services enable
compute.googleapis.com cloudbilling.googleapis.com billingbudgets.googleapis.com
iam.googleapis.com cloudresourcemanager.googleapis.com`
40
41      `provision.sh` (§5) is idempotent and re-runs all of step (3) + the bucket/SA/budget.
42
43      ---
44
45      ## 2. ? GPU quota ? the one remaining owner action
46
47      New GCP projects start at **`GPUS_ALL_REGIONS` = 0**, the *global* cap that gates GPU
creation even
48      when a regional per-type quota is non-zero. Confirmed 2026-06-20:
49
50      ```
51      asia-south1  NVIDIA_T4_GPUS = 1   NVIDIA_L4_GPUS = 1   (regional: OK)
52      GLOBAL       GPUS_ALL_REGIONS = 0  ? BLOCKS the VM
53      ```
54
55      **Owner:** IAM & Admin ? **Quotas** ? filter **"GPUs (all regions)"** ? Edit ? request
**1** ? submit.
56      (For a single T4 this is usually approved in minutes; sometimes a short review.) Re-
check:
57
58      ```bash
59      gcloud compute project-info describe --project gen-lang-client-0306026826 \
60        --flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" | grep
GPUS_ALL_REGIONS
61      ```
62
63      When that reads `1`, run §3.
64
65      > **? Fast path ? Spot T4 likely skips the wait.** `GPUS_ALL_REGIONS` gates **on-
demand** GPUs;
66      > **preemptible/Spot** GPUs are gated by the *regional* `PREEMPTIBLE_NVIDIA_T4_GPUS`
```

```
quota, which is
67      > **already 1** in asia-south1. So a **Spot** T4 (`--provisioning-model=SPOT`, §3
variant) should
68      > create **without** the quota increase ? verify by trying it. Spot is also ~60-70%
cheaper. Tradeoff:
69      > it can be preempted; the WASB detector's resumable cache (checkpoints every N windows)
makes a
70      > re-run cheap, and Gen-0 training is minutes. Recommended for the first real run;
switch to an
71      > on-demand VM (after the quota grant) only if preemption proves disruptive.
72
73      ---
74
75      ## 3. Create the GPU VM (after quota ? 1)
76
77      T4 is plenty for WASB + Gen-0. A local SSD gives a fast `/scratch` for frame extraction.
78
79      ```bash
80      gcloud compute instances create rally-gpu \
81        --project=gen-lang-client-0306026826 \
82        --zone=asia-south1-a \
83        --machine-type=n1-standard-8 \
84        --accelerator=type=nvidia-tesla-t4,count=1 \
85        --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
86        --image-family=ubuntu-2404-lts-amd64 --image-project=ubuntu-os-cloud \
87        --boot-disk-size=120GB --boot-disk-type=pd-balanced \
88        --local-ssd=interface=NVME \
89        --service-account=rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com \
90        --scopes=cloud-platform
91      ```
92
93      **Spot variant (no GPUS_ALL_REGIONS wait ? see §2 fast path):** same command with
94      `--provisioning-model=SPOT` (replacing `STANDARD`). Add `--instance-termination-
action=STOP` to keep
95      the disk on preemption so a re-run resumes. Cheaper, can be preempted.
96
97      > If `asia-south1-a` reports no T4 capacity, try `-b` / `-c`. The Ubuntu image needs the
NVIDIA
98      > driver: either add `--metadata=install-nvidia-driver=True` style startup, or after SSH
run GCP's
99      > driver installer, **or** use a Deep-Learning-VM image (`--image-family=common-
cu124-ubuntu-2204`
100     > `--image-project=deeplearning-platform-release`) which ships the driver. Confirm
`nvidia-smi` works
101     > before §4.
102
103     SSH: `gcloud compute ssh rally-gpu --zone=asia-south1-a`.
104
105     ---
106
107     ## 4. Port the repo + WASB env onto the box
108
109     The `deploy/digitalocean/` scripts are **provider-agnostic** (they detect the local NVMe
/ install
110     torch); reuse them on GCP:
111
112     ```bash
113     # from your machine ? ship the repo:
114     git archive --format=tar.gz -o /tmp/rally.tgz HEAD
115     gcloud compute scp /tmp/rally.tgz rally-gpu:/root/ --zone=asia-south1-a
116     gcloud compute ssh rally-gpu --zone=asia-south1-a -- 'mkdir -p ~/rally && tar -xzf
~/rally.tgz -C ~/rally'
117
118     # on the box:
119     cd ~/rally
```

```
    sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch   # detects the local NVMe
    ./deploy/digitalocean/bootstrap.sh --gpu                                    # venv + torch cu124 (confirm cuda True)
    . .venv/bin/activate && pip install -e .[storage-gcs]                       # GCS backend
    bash deploy/digitalocean/gpu_setup.sh                                       # native `wasb` conda env + WASB-SBDT

    # secrets (NOT in repo): the GCS SA JSON + a rotated Gemini key
    gcloud compute scp ~/.gcp/rally-worker-sa.json rally-gpu:/root/.gcp/sa.json --zone=asia-south1-a
    # on the box: export GOOGLE_APPLICATION_CREDENTIALS=/root/.gcp/sa.json
    mkdir -p /root/.keys && printf '%s' '<ROTATED_GEMINI_KEY>' > /root/.keys/GEMINI_API_KEY && chmod 600 /root/.keys/GEMINI_API_KEY
    ```

    WASB weights (`wasb_badminton_best.pth.tar`) must be present at
    `~/models/WASB-SBDT/pretrained_weights/` (gpu_setup.sh does NOT fetch them ? stage from Drive/bucket).
    Point the native runner at them: `export WASB_WEIGHTS=~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar`
    and `export WASB_PYTHON=$(conda run -n wasb which python)` (or the env's python path). See
    `docs/TRACKNET_WSL_SETUP.md` for the WASB env. **WASB-C3:** weight provenance is an open legal item
    before sharing any trajectories/model externally ? internal extraction + Gen-0 is fine.

    ## 5. Stage the corpus + run the first REAL train/infer

    Config for GCS (no secrets in config ? auth via `GOOGLE_APPLICATION_CREDENTIALS`):

    ```json
    { "sport": "badminton",
      "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "native" },
      "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826" } }
}
    ```

    ```bash
    # stage the 7 Done videos + 7 labels into the bucket (from the co-located box ? fast):
    #   videos -> gs://khelsutra-rally-corpus/videos/   labels -> gs://khelsutra-rally-corpus/labels/
    # (owner handles the actual copy; see "Upload options" below)

    # FIRST REAL training (real WASB trajectories, native runner):
    RALLY_DETECTOR_IMPL=native python -m backend.worker train \
      --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
      --version 0.1.0 --trajectory-source detector --segmenter wasb_hybrid --config config.json

    # FIRST REAL inference on the held-out Video 1 (GX010135_proxy.mp4, no labels):
    python -m backend.worker infer \
      --video-uri gcs://khelsutra-rally-corpus/videos/GX010135_proxy.mp4 \
      --out-uri  gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json
    ```

    ### Upload options (how to get a video in + run inference)
    The worker resolves `--video-uri` via the storage layer ? pick whichever fits:
    - **GCS bucket (recommended, co-located):** `gcs://khelsutra-rally-corpus/videos/clip.mp4`. Upload with
      `gcloud storage cp clip.mp4 gs://khelsutra-rally-corpus/videos/`.
    - **S3 / DO Spaces / R2:** `s3://bucket/clip.mp4` (install `.[storage-s3]`, creds in `~/.aws/credentials`).
```
```

```
171     - **Local upload:** scp the file to the box and pass a bare path /
`local:///root/clip.mp4` (identity passthrough, no copy).
172     - **Google Drive:** `gdrive://` is not yet a first-class backend (raises a clear error).
Use
173       `gdown <file-id> -O clip.mp4` to pull to local, then pass the local path. *(A first-
class `gdrive://`
174       backend is a tracked follow-up.)*
175
176     ---
177
178     ## 6. Switch off between sessions (cost control)
179
180     ```bash
181     bash deploy/gcp/teardown.sh stop     # stop the VM (keep disk; ~$0 compute) ? fastest
resume
182     bash deploy/gcp/teardown.sh delete   # delete the VM (no disk cost; full re-port to
resume)
183     bash deploy/gcp/teardown.sh all      # delete VM + SA + budget (keeps the bucket/data)
184     ```
185
186     The **bucket is never auto-deleted** (it holds the corpus). Delete it explicitly only
when done:
187     `gcloud storage rm -r gs://khelsutra-rally-corpus`.
188
```

20. assistant (2026-06-20T09:40:30.749Z)

Now let me read the `docs/SETUP_NEW_MACHINE.md` §7 section and the config model for gdrive.

21. user (2026-06-20T09:40:32.168Z)

```
185       "region": "sgp1"
186       // AWS S3 : omit endpoint_url; set region (e.g. ap-south-1)
187       // R2     : endpoint_url https://<account>.r2.cloudflarestorage.com ; region "auto"
188       // MinIO  : endpoint_url http://host:9000 ; region us-east-1 ; "addressing_style":
"path"
189     }}
190     ```
191     **Run** (secret-free flavor; use `--segmenter gemini` for real detection):
192     ```bash
193     env -u GEMINI_API_KEY RALLY_DETECTOR_IMPL=stub python -m backend.worker infer \
194       --video-uri s3://<bucket>/videos/clip.mp4 --out-uri s3://<bucket> \
195       --segmenter wasb_hybrid --set indexing.skip_ai_handoff=true
196     ```
197
198     ### 7b. Google Cloud Storage
199
200     `pip install -e .[storage-gcs]`. **Credentials** ? a service-account JSON (SA with
*Storage Object Admin*):
201     ```bash
202     export GOOGLE_APPLICATION_CREDENTIALS=/path/to/sa.json
203     ```
204     **Config** ? `storage.gcs.project` (no endpoint ? default `googleapis.com`):
205     ```jsonc
206     "storage": { "backend": "local", "gcs": { "project": "<gcp-project-id>" } }
207     ```
208     Same `worker infer` command with `gcs://<bucket>/?` URIs. *(On-box test with no GCP
creds:
209     `pip install gcp-storage-emulator && gcp-storage-emulator start --port 9023 --in-memory`
+
210     `export STORAGE_EMULATOR_HOST=http://127.0.0.1:9023`.)*
211
212     ### 7c. Google Drive *(corpus source ? follow-up backend)*
213
214     `gdrive://` is recognised; the backend is a focused follow-up (for the read-only golden
```

```
corpus).
215
216     **Bucket layout:** `videos/ labels/ manifest.json weights/ outputs/<video_id>/` (see
`docs/DECOUPLED_COMPUTE/04`).
217
218     ---
219
220     ## 8. Run training (Gen-0 rally head)
221
222     CPU-runnable, minutes ? but it needs a **labeled corpus** (videos + `.rallies.csv`
labels + a manifest),
223     which a fresh box doesn't have. Bring it via §7's bucket (`worker train`, pending) or
`scp` it up, then:
224     ```bash
225     python -m training.gen0.harness --manifest <manifest.json> --floor <baseline> --version
0.1.0
226     ```
227     The harness runs train ? eval ? ship-gate and emits a versioned weight bundle. *(The
Gen-2 VLM is a
228     separate, GPU-heavy, separately-budgeted effort ? out of scope here.)*
229
```

22. user (2026-06-20T09:40:32.973Z)

```
280         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
281         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
282         # contain upload_dir or /api/process will reject the uploaded paths.
283         upload_dir: str = "output/uploads"
284         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
285         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
286         max_upload_mb: int = 4096
287         max_part_mb: int = 95
288         allowed_upload_extensions: List[str] = ["mp4", "mov"]
289
290
291     class StorageConfig(BaseModel):
292         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
293         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
294         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google
Drive
295         (follow-up). Per-backend settings are loose dicts passed to the backend constructor
?
296         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
297         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
298         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
299         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
300         output_root: str = ""
301         local: dict[str, Any] = Field(default_factory=dict)
302         s3: dict[str, Any] = Field(default_factory=dict)
303         gcs: dict[str, Any] = Field(default_factory=dict)
304         gdrive: dict[str, Any] = Field(default_factory=dict)
305
306
307     class AppConfig(BaseModel):
308         """Root configuration object.
309
```

23. assistant (2026-06-20T09:40:39.388Z)

I have a full picture now. Let me set up a clean branch off `origin/master` per the branching-safety protocol.

24. user (2026-06-20T09:40:40.847Z)

```
55ae408 docs(decoupled): GCP pivot runbook + DO?GCP ADR-010 (deploy/gcp + handoff) (#257)
---
branch 'feat/gdrive-storage-backend' set up to track 'origin/master'.
Switched to a new branch 'feat/gdrive-storage-backend'
---NOW---
feat/gdrive-storage-backend
55ae408 docs(decoupled): GCP pivot runbook + DO?GCP ADR-010 (deploy/gcp + handoff) (#257)
```

25. assistant (2026-06-20T09:40:53.810Z)

Let me confirm the files on this branch match what I read (I switched branches), then write the implementation.

26. user (2026-06-20T09:40:55.873Z)

```
EXIT: no diff means identical to what I read
backend/storage/gdrive.py
deploy/gcp/README.md
```

27. assistant (2026-06-20T09:41:08.574Z)

Files match. Now I'll implement `GDriveStorage`.

28. assistant (2026-06-20T09:41:29.809Z)

```
"""Google Drive storage backend ? the owner's initial-testing store (2 TB via Google AI Plus).

Drive is a genuinely *second-class* backend: it has no real path namespace (objects are
addressed by an opaque **file-id**, names can collide) and it cannot mint presigned URLs.
We model that honestly with an **id-based URI**:

    gdrive://<file-id>                -> one file   (fetch_to_local / stat / exists / delete)
    gdrive://<folder-id>              -> a folder   (list its immediate children)
    gdrive://<folder-id>/<title>      -> put target (create ``<title>`` inside ``<folder-id>``)

So ``ref.bucket`` always carries a Drive **id** (a file-id for object ops, a folder-id for
``list``/``put``); ``ref.key`` is only consulted by ``put`` (its basename becomes the new file's
title). ``list``/``put`` return refs whose URI is ``gdrive://<new-file-id>`` ? the only address
you can reliably fetch back.

Uses ``pydrive2``, imported lazily so ``import backend.storage`` never pulls it; install with
the
``storage-gdrive`` extra. Auth mirrors the GCS backend: credentials come from a **service-
account
JSON named by the environment** (``GDRIVE_SERVICE_ACCOUNT_JSON`` or, falling back, the shared
``GOOGLE_APPLICATION_CREDENTIALS`` ADC path) ? never from config. A ``drive`` object may be
injected for tests.
"""
from __future__ import annotations

import os
from datetime import datetime
from typing import Any, Iterator, Optional

from backend.storage.base import StorageBackend
from backend.storage.ref import StorageRef, StorageStat


def _parse_drive_date(value: Any) -> Optional[datetime]:
    """Parse a Drive v2 RFC-3339 timestamp (``2026-06-20T12:34:56.789Z``) to a ``datetime``;
    return ``None`` for anything unparseable (``modified`` is optional metadata)."""
    if not value:
```

```
        return None
    s = str(value).replace("Z", "+00:00")
    try:
        return datetime.fromisoformat(s)
    except ValueError:
        return None


class GDriveStorage(StorageBackend):
    def __init__(self, *, drive: Any = None, **_ignored) -> None:
        if drive is not None:
            self._drive = drive
            return
        # Lazy: pydrive2 + the SA JSON are touched only when a gdrive:// URI is actually used.
        creds = (os.environ.get("GDRIVE_SERVICE_ACCOUNT_JSON")
                 or os.environ.get("GOOGLE_APPLICATION_CREDENTIALS"))
        if not creds:
            raise RuntimeError(
                "Google Drive needs a service-account JSON: set GDRIVE_SERVICE_ACCOUNT_JSON "
                "(or the shared GOOGLE_APPLICATION_CREDENTIALS) to its path. Credentials are "
                "never read from config."
            )
        from pydrive2.auth import GoogleAuth  # type: ignore  # lazy: only on a gdrive:// URI
        from pydrive2.drive import GoogleDrive  # type: ignore
        gauth = GoogleAuth(settings={
            "client_config_backend": "service",
            "service_config": {"client_json_file_path": creds},
        })
        gauth.ServiceAuth()
        self._drive = GoogleDrive(gauth)

    @staticmethod
    def _id(ref: StorageRef) -> str:
        """The Drive id a ref points at (a file-id for object ops, a folder-id for list/put)."""
        if not ref.bucket:
            raise ValueError(f"gdrive:// URI needs a Drive id (file or folder): {ref.uri!r}")
        return ref.bucket

    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
        if dst is None:
            raise ValueError("GDriveStorage.fetch_to_local requires a dst path")
        dst = str(dst)
        if os.path.exists(dst) and not overwrite:
            return dst
        os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
        self._drive.CreateFile({"id": self._id(ref)}).GetContentFile(dst)
        return dst

    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
        folder_id = self._id(ref)
        title = ref.name or os.path.basename(str(local_path))
        meta = {"title": title, "parents": [{"id": folder_id}]}
        if content_type:
            meta["mimeType"] = content_type
        f = self._drive.CreateFile(meta)
        f.SetContentFile(str(local_path))
        f.Upload()
        new_id = f.get("id") or folder_id
        return StorageRef("gdrive", new_id, title, f"gdrive://{new_id}")

    def exists(self, ref) -> bool:
        f = self._drive.CreateFile({"id": self._id(ref)})
        try:
            f.FetchMetadata()
        except Exception:  # noqa: BLE001 - any pydrive2 error means "not found / unreachable"
```

```
            return False
        return not f.get("labels", {}).get("trashed", False)

    def stat(self, ref) -> StorageStat:
        f = self._drive.CreateFile({"id": self._id(ref)})
        f.FetchMetadata(fetch_all=True)
        return StorageStat(
            size=int(f.get("fileSize") or 0),
            modified=_parse_drive_date(f.get("modifiedDate")),
            etag=f.get("md5Checksum"),
            content_type=f.get("mimeType"),
        )

    def list(self, prefix) -> Iterator[StorageRef]:
        folder_id = self._id(prefix)
        query = {"q": f"'{folder_id}' in parents and trashed=false"}
        for child in self._drive.ListFile(query).GetList():
            cid = child["id"]
            yield StorageRef("gdrive", cid, child.get("title", ""), f"gdrive://{cid}")

    def delete(self, ref) -> None:
        self._drive.CreateFile({"id": self._id(ref)}).Delete()
```

29. user (2026-06-20T09:41:29.833Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

30. assistant (2026-06-20T09:41:39.276Z)

```
Now update the package docstrings that still call Drive a "follow-up" and fix the `ref.py` URI
doc to the id-based model.
```

31. user (2026-06-20T09:41:40.717Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\__init__.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

32. user (2026-06-20T09:41:42.924Z)

```
<tool_use_error>String to replace not found in file.
String:     gcs://<bucket>/<key>         -> gcs       (gs:// accepted, normalized to gcs://)
    gdrive://<folderId>/<path>  -> gdrive   (follow-up backend)</tool_use_error>
```

33. user (2026-06-20T09:41:47.605Z)

```
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<folderId>/<path>  -> gdrive   (follow-up backend)
10
```

34. user (2026-06-20T09:41:50.770Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\ref.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

35. assistant (2026-06-20T09:42:36.562Z)

```
Now the tests. First the imports and module docstring:
```

36. user (2026-06-20T09:42:40.799Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

37. assistant (2026-06-20T09:42:44.249Z)

Now replace the stale "not implemented" test:

38. user (2026-06-20T09:42:48.251Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

39. assistant (2026-06-20T09:42:51.206Z)

Now add the GDrive fake client + tests after the GCS section, before the lazy-import section:

40. user (2026-06-20T09:43:13.340Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

41. assistant (2026-06-20T09:43:26.342Z)

Now the docs. Let me update `pyproject.toml`, the config-model docstring, and the two doc
sections.

42. user (2026-06-20T09:43:28.712Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\pyproject.toml has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

43. user (2026-06-20T09:43:30.754Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

44. assistant (2026-06-20T09:43:33.028Z)

Now `deploy/gcp/README.md` §5 Upload options:

45. user (2026-06-20T09:43:38.329Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\gcp\README.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

46. assistant (2026-06-20T09:43:41.285Z)

Now `docs/SETUP_NEW_MACHINE.md` §7c:

47. user (2026-06-20T09:43:48.359Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\SETUP_NEW_MACHINE.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

48. assistant (2026-06-20T09:43:52.411Z)

Let me fix the two stale references in `docs/DECOUPLED_COMPUTE/HANDOFF.md` for accuracy.

49. user (2026-06-20T09:43:53.343Z)

```
20        (reuse `deploy/digitalocean/*` ? provider-agnostic) ? stage corpus ? run the FIRST
```

```
real
21        `worker train --trajectory-source detector` + `worker infer` on held-out Video 1.
22      - **Upload/run options** for inference: GCS bucket URI (recommended), local path/scp, or
GDrive via
23        `gdown`?local (`gdrive://` backend is a tracked follow-up). See `deploy/gcp/README.md`
§5.
24
25      ---
26
27      ## TL;DR ? where we are (pre-2026-06-20-update; M3.5 now done per above)
28
29      The **entire bucket-driven `train ? infer` pipeline runs end-to-end on a CPU box**, with
the GPU **mocked**
30      (deterministic stub). Validated on a DigitalOcean droplet reading/writing **DO Spaces**,
driven remotely over
31      SSH. Everything is merged to `master` (PRs #246?#254); the suite is **936 green**.
32
33      **The ONE remaining piece for *real* (non-mock) training** is the **Linux-native WASB
`DetectorRunner` (M3.5)**
34      running on a **GPU box** ? it produces *real* shuttle trajectories instead of the
synthetic stub. Once it
35      exists, the **same `worker train` / `worker infer` commands run unchanged**; only the
trajectory source flips.
36
37      ## What's built (all merged to master)
38
39      | PR | What | Key files |
40      |---|---|---|
41      | #249 | Storage layer ? `local` + **S3 (AWS/DO Spaces/R2/MinIO)** + GCS, URI-driven;
gdrive stub | `backend/storage/` |
42      | #251, #254 | **Worker** `python -m backend.worker {infer\|train}` ? pull from storage
? pipeline ? push | `backend/worker/__main__.py` |
43      | #247 | CPU-mock detector seam (`RALLY_DETECTOR_IMPL=stub` /
`indexing.detector_impl=stub`) | `backend/pipeline/detectors/stub_runner.py` |
```

50. user (2026-06-20T09:43:58.806Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DECOUPLED_COMPUTE\HANDOFF.md
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

51. user (2026-06-20T09:44:00.677Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DECOUPLED_COMPUTE\HANDOFF.md
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

52. assistant (2026-06-20T09:44:08.674Z)

Now let me verify. First the storage tests in isolation, plus check the lint/type config to match CI exactly.

53. user (2026-06-20T09:44:11.649Z)

```
............................                                                [100%]
28 passed in 0.37s
```

54. user (2026-06-20T09:44:11.663Z)

```
1       name: CI
2
3       on:
4         push:
5           branches: [master]
6         pull_request:
7
```

```
8       permissions:
9         contents: read
10
11      concurrency:
12        group: ci-${{ github.ref }}
13        cancel-in-progress: true
14
15      jobs:
16        # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17        # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18        # tracked .py, including scripts/ and training/, which the backend-only
19        # py_compile smoke sweep does not reach.
20        lint:
21          name: Ruff lint
22          runs-on: ubuntu-latest
23          steps:
24            - uses: actions/checkout@v6
25            - uses: actions/setup-python@v6
26              with:
27                python-version: "3.13"
28            - name: Install ruff
29              run: python -m pip install ruff
30            - name: Ruff check
31              run: ruff check . --output-format github
32
33        # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34        # per-module quarantine list ? ratchet by removing entries as modules clean
35        # up. Typed core deps are installed so mypy sees their real signatures
36        # (without them ignore_missing_imports would Any-stub half the checks).
37        typecheck:
38          name: Mypy (lenient baseline)
39          runs-on: ubuntu-latest
40          steps:
41            - uses: actions/checkout@v6
42            - uses: actions/setup-python@v6
43              with:
44                python-version: "3.13"
45            - name: Install mypy + typed core deps
46              run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
genai numpy
47            - name: Mypy
48              run: mypy backend training
49
50        # Fast guard against the 7fbbeb0 class of breakage: a SyntaxError or
51        # module-level import error shipping while the suite stays green.
52        # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
53        # on machines without the GPU stack (test_import_smoke stubs what's absent).
54        import-smoke:
55          name: Syntax sweep + import smoke (no GPU stack)
56          runs-on: ubuntu-latest
57          steps:
58            - uses: actions/checkout@v6
59            - uses: actions/setup-python@v6
60              with:
61                python-version: "3.13"
62            - name: Install minimal deps
63              run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
64            - name: Run import smoke tests
65              run: python -m pytest tests/test_import_smoke.py -v
66
67        full-suite:
68          name: Full test suite (offline, mocked)
69          runs-on: ubuntu-latest
70          steps:
```

```
71            - uses: actions/checkout@v6
72            - uses: actions/setup-python@v6
73              with:
74                python-version: "3.13"
75                cache: pip
76            - name: System libs for opencv-python
77              run: |
78                sudo apt-get update
79                sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
80            - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81              # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82              # need an out-of-band index that PEP 621 can't express. Keep installing
83              # them separately here, before the editable install pulls the rest.
84              run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85            - name: Install package (editable) + dev tooling
86              # Editable PEP 621 install: runtime deps + the `dev` group
87              # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88              run: python -m pip install -e .[dev]
89            # obsreport (private, soft dep) is absent here: reporting tests skip via
90            # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91            # is therefore calibrated against THIS lane's number (local runs with
92            # obsreport installed measure a little higher).
93            # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94            # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95            # without an owner decision.
96            - name: Run full suite (with coverage floor)
97              run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
99        # Cross-OS determinism guard for the committed golden regression fixtures
100       # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101       # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102       # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103       # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104       # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105       golden-crossos:
106         name: Golden fixtures cross-OS (${{ matrix.os }})
107         strategy:
108           fail-fast: false
109           matrix:
110             os: [ubuntu-latest, windows-latest, macos-latest]
111         runs-on: ${{ matrix.os }}
112         steps:
113           - uses: actions/checkout@v6
114           - uses: actions/setup-python@v6
115             with:
116               python-version: "3.13"
117           - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118             run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
119           - name: Golden-fixtures characterization (cross-OS determinism guard)
120             # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
121             # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
122             # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
123             run: |
124               python -m backend.eval.golden_fixtures --check
125               python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
126
```

55. assistant (2026-06-20T09:44:17.163Z)

All 28 storage tests pass. Now ruff + mypy (matching CI), run in parallel:

56. user (2026-06-20T09:44:19.252Z)

```
Exit code 1
artifacts\landing\make_og.py:23:1: E731 Do not assign a `lambda` expression, use a `def`
artifacts\landing\make_og.py:24:1: E731 Do not assign a `lambda` expression, use a `def`
Found 2 errors.
No fixes available (2 hidden fixes can be enabled with the `--unsafe-fixes` option).
```

57. user (2026-06-20T09:44:21.821Z)

```
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 122 source files
```

58. assistant (2026-06-20T09:44:29.370Z)

mypy is clean. The 2 ruff errors are in `artifacts/landing/make_og.py` ? an untracked local file (the `?? artifacts/` from git status), not part of my change and absent in CI. Let me confirm my touched files are ruff-clean:

59. user (2026-06-20T09:44:32.180Z)

```
All checks passed!
--- confirm artifacts/ is untracked, not mine ---
?? artifacts/
```

60. assistant (2026-06-20T09:44:36.623Z)

My files are clean. Now the full suite, exactly as CI runs it (with the coverage floor):

61. user (2026-06-20T09:46:35.321Z)

```
backend\pipeline\validators\base.py                  39      3    92%
backend\providers\__init__.py                        16      1    94%
backend\providers\base.py                             9      1    89%
backend\providers\gemini.py                          99     16    84%
backend\reporting\__init__.py                        26      2    92%
backend\reporting\harvest.py                        111      8    93%
backend\reporting\run_signals.py                     14      0   100%
backend\results.py                                   27      0   100%
backend\sports\__init__.py                           17      1    94%
backend\sports\badminton.py                          13      1    92%
backend\sports\base.py                               12      3    75%
backend\storage\__init__.py                          37      4    89%
backend\storage\base.py                              14      0   100%
backend\storage\gcs.py                               41      4    90%
backend\storage\gdrive.py                            71      9    87%
backend\storage\local.py                             49      4    92%
backend\storage\ref.py                               37      0   100%
backend\storage\s3.py                                47      5    89%
backend\tools\__init__.py                             0      0   100%
backend\tools\export.py                              75      2    97%
backend\tools\rally_slicer.py                       104     47    55%
backend\utils\__init__.py                             0      0   100%
```

```
backend\utils\geometry.py                              29      0   100%
backend\utils\hashing.py                                9      0   100%
backend\utils\logging_setup.py                         11      0   100%
backend\utils\paths.py                                 27      3    89%
backend\utils\run_telemetry.py                        126     12    90%
backend\utils\video.py                                 25      3    88%
backend\worker\__init__.py                              0      0   100%
backend\worker\__main__.py                            246     31    87%
training\__init__.py                                    0      0   100%
training\gen0\__init__.py                               0      0   100%
training\gen0\harness.py                              108     11    90%
-----------------------------------------------------------------------
TOTAL                                                8903   1448    84%
Required test coverage of 70% reached. Total coverage: 83.74%
1003 passed, 7 warnings in 113.46s (0:01:53)
=========================================================================
 RESULT: PASS - all tests green
=========================================================================
```

62. assistant (2026-06-20T09:46:52.826Z)

Full suite green: **1003 passed**, coverage **83.74%** (?70 floor), and `gdrive.py` itself at 87% (the 9 uncovered lines are the real `pydrive2`/`ServiceAuth` branch that can't run offline ? same as the GCS/S3 real-client branches). Let me review the complete diff before committing.

63. user (2026-06-20T09:46:54.887Z)

```
warning: in the working copy of 'backend/storage/gdrive.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'deploy/gcp/README.md', LF will be replaced by CRLF the next
time Git touches it
 backend/config/models.py          |   2 +-
 backend/storage/__init__.py       |   2 +-
 backend/storage/gdrive.py         | 133 +++++++++++++++++++++++++++++-------
 backend/storage/ref.py            |   2 +-
 deploy/gcp/README.md              |   9 ++-
 docs/DECOUPLED_COMPUTE/HANDOFF.md |   6 +-
 docs/SETUP_NEW_MACHINE.md         |  20 +++++-
 pyproject.toml                    |   2 +-
 tests/test_storage.py             | 144 ++++++++++++++++++++++++++++++++++++++--
 9 files changed, 274 insertions(+), 46 deletions(-)
=====FULL DIFF (non-doc code)=====
warning: in the working copy of 'backend/storage/gdrive.py', LF will be replaced by CRLF the
next time Git touches it
diff --git a/backend/storage/gdrive.py b/backend/storage/gdrive.py
index eae21ba..1c252ad 100644
--- a/backend/storage/gdrive.py
+++ b/backend/storage/gdrive.py
@@ -1,41 +1,122 @@
-"""Google Drive backend ? placeholder for a focused follow-up PR.
+"""Google Drive storage backend ? the owner's initial-testing store (2 TB via Google AI Plus).

-Drive is the owner's initial-testing store (2 TB via Google AI Plus), but it is a genuinely
-*second-class* backend: a file-id model (no real path namespace) and no presigned URLs, plus
-OAuth (``drive.file``) vs service-account auth choices that need the real Drive to validate.
-It is therefore implemented in a dedicated follow-up (via ``pydrive2``, the ``storage-gdrive``
-extra) rather than guessed at here ? keeping the first-class S3/GCS path clean and tested.
+Drive is a genuinely *second-class* backend: it has no real path namespace (objects are
+addressed by an opaque **file-id**, names can collide) and it cannot mint presigned URLs.
+We model that honestly with an **id-based URI**:

-Recognising the ``gdrive://`` scheme now (with a clear error) beats a confusing
-``unknown backend`` later.
+    gdrive://<file-id>                -> one file   (fetch_to_local / stat / exists / delete)
+    gdrive://<folder-id>              -> a folder   (list its immediate children)
```

```
+    gdrive://<folder-id>/<title>       -> put target (create ``<title>`` inside ``<folder-
id>``)
+
+So ``ref.bucket`` always carries a Drive **id** (a file-id for object ops, a folder-id for
+``list``/``put``); ``ref.key`` is only consulted by ``put`` (its basename becomes the new
file's
+title). ``list``/``put`` return refs whose URI is ``gdrive://<new-file-id>`` ? the only address
+you can reliably fetch back.
+
+Uses ``pydrive2``, imported lazily so ``import backend.storage`` never pulls it; install with
the
+``storage-gdrive`` extra. Auth mirrors the GCS backend: credentials come from a **service-
account
+JSON named by the environment** (``GDRIVE_SERVICE_ACCOUNT_JSON`` or, falling back, the shared
+``GOOGLE_APPLICATION_CREDENTIALS`` ADC path) ? never from config. A ``drive`` object may be
+injected for tests.
 """
 from __future__ import annotations

+import os
+from datetime import datetime
+from typing import Any, Iterator, Optional
+
 from backend.storage.base import StorageBackend
+from backend.storage.ref import StorageRef, StorageStat
+
+
+def _parse_drive_date(value: Any) -> Optional[datetime]:
+    """Parse a Drive v2 RFC-3339 timestamp (``2026-06-20T12:34:56.789Z``) to a ``datetime``;
+    return ``None`` for anything unparseable (``modified`` is optional metadata)."""
+    if not value:
+        return None
+    s = str(value).replace("Z", "+00:00")
+    try:
+        return datetime.fromisoformat(s)
+    except ValueError:
+        return None


 class GDriveStorage(StorageBackend):
-    def __init__(self, **_ignored) -> None:
-        raise NotImplementedError(
-            "The Google Drive backend (gdrive://) is not implemented yet ? it lands in a "
-            "follow-up PR (pydrive2 + the 'storage-gdrive' extra), validated against the "
-            "owner's Drive. Use local://, s3:// or gcs:// for now."
-        )
+    def __init__(self, *, drive: Any = None, **_ignored) -> None:
+        if drive is not None:
+            self._drive = drive
+            return
+        # Lazy: pydrive2 + the SA JSON are touched only when a gdrive:// URI is actually used.
+        creds = (os.environ.get("GDRIVE_SERVICE_ACCOUNT_JSON")
+                 or os.environ.get("GOOGLE_APPLICATION_CREDENTIALS"))
+        if not creds:
+            raise RuntimeError(
+                "Google Drive needs a service-account JSON: set GDRIVE_SERVICE_ACCOUNT_JSON "
+                "(or the shared GOOGLE_APPLICATION_CREDENTIALS) to its path. Credentials are "
+                "never read from config."
+            )
+        from pydrive2.auth import GoogleAuth  # type: ignore  # lazy: only on a gdrive:// URI
+        from pydrive2.drive import GoogleDrive  # type: ignore
+        gauth = GoogleAuth(settings={
+            "client_config_backend": "service",
+            "service_config": {"client_json_file_path": creds},
+        })
```

```
+        gauth.ServiceAuth()
+        self._drive = GoogleDrive(gauth)

-    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:  # pragma: no cover
-        raise NotImplementedError
+    @staticmethod
+    def _id(ref: StorageRef) -> str:
+        """The Drive id a ref points at (a file-id for object ops, a folder-id for
list/put)."""
+        if not ref.bucket:
+            raise ValueError(f"gdrive:// URI needs a Drive id (file or folder): {ref.uri!r}")
+        return ref.bucket

-    def put(self, local_path, ref, *, content_type=None):  # pragma: no cover
-        raise NotImplementedError
+    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
+        if dst is None:
+            raise ValueError("GDriveStorage.fetch_to_local requires a dst path")
+        dst = str(dst)
+        if os.path.exists(dst) and not overwrite:
+            return dst
+        os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
+        self._drive.CreateFile({"id": self._id(ref)}).GetContentFile(dst)
+        return dst

-    def exists(self, ref) -> bool:  # pragma: no cover
-        raise NotImplementedError
+    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
+        folder_id = self._id(ref)
+        title = ref.name or os.path.basename(str(local_path))
+        meta = {"title": title, "parents": [{"id": folder_id}]}
+        if content_type:
+            meta["mimeType"] = content_type
+        f = self._drive.CreateFile(meta)
+        f.SetContentFile(str(local_path))
+        f.Upload()
+        new_id = f.get("id") or folder_id
+        return StorageRef("gdrive", new_id, title, f"gdrive://{new_id}")

-    def stat(self, ref):  # pragma: no cover
-        raise NotImplementedError
+    def exists(self, ref) -> bool:
+        f = self._drive.CreateFile({"id": self._id(ref)})
+        try:
+            f.FetchMetadata()
+        except Exception:  # noqa: BLE001 - any pydrive2 error means "not found / unreachable"
+            return False
+        return not f.get("labels", {}).get("trashed", False)
+
+    def stat(self, ref) -> StorageStat:
+        f = self._drive.CreateFile({"id": self._id(ref)})
+        f.FetchMetadata(fetch_all=True)
+        return StorageStat(
+            size=int(f.get("fileSize") or 0),
+            modified=_parse_drive_date(f.get("modifiedDate")),
+            etag=f.get("md5Checksum"),
+            content_type=f.get("mimeType"),
+        )

-    def list(self, prefix):  # pragma: no cover
-        raise NotImplementedError
+    def list(self, prefix) -> Iterator[StorageRef]:
+        folder_id = self._id(prefix)
+        query = {"q": f"'{folder_id}' in parents and trashed=false"}
+        for child in self._drive.ListFile(query).GetList():
```

```
+            cid = child["id"]
+            yield StorageRef("gdrive", cid, child.get("title", ""), f"gdrive://{cid}")

-    def delete(self, ref) -> None:  # pragma: no cover
-        raise NotImplementedError
+    def delete(self, ref) -> None:
+        self._drive.CreateFile({"id": self._id(ref)}).Delete()
```

64. assistant (2026-06-20T09:47:10.528Z)

The diff is clean (LF?CRLF warnings are just Windows autocrlf, matching the other backend files). Per branching-safety, let me re-verify HEAD before staging explicit paths and committing.

65. user (2026-06-20T09:47:11.851Z)

```
branch: feat/gdrive-storage-backend
HEAD: 55ae408 docs(decoupled): GCP pivot runbook + DO?GCP ADR-010 (deploy/gcp + handoff) (#257)
--- commits ahead of origin/master (should be EMPTY before commit) ---
```

66. assistant (2026-06-20T09:47:14.352Z)

HEAD is at `origin/master` with no sibling commits. Staging explicit paths only (never `-A`) and committing:

67. user (2026-06-20T09:47:16.693Z)

```
warning: in the working copy of 'backend/storage/gdrive.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'deploy/gcp/README.md', LF will be replaced by CRLF the next
time Git touches it
--- staged ---
M  backend/config/models.py
M  backend/storage/__init__.py
M  backend/storage/gdrive.py
M  backend/storage/ref.py
M  deploy/gcp/README.md
M  docs/DECOUPLED_COMPUTE/HANDOFF.md
M  docs/SETUP_NEW_MACHINE.md
M  pyproject.toml
M  tests/test_storage.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
--- sanity: nothing else staged (artifacts/ + KHELSUTRA doc stay untracked) ---
```

68. assistant (2026-06-20T09:47:19.614Z)

Exactly my 9 files staged; `artifacts/` and the KHELSUTRA doc remain untracked. Committing:

69. user (2026-06-20T09:47:27.357Z)

```
[feat/gdrive-storage-backend 5fa7f30] feat(storage): implement Google Drive backend (gdrive://
as first-class worker input)
 9 files changed, 274 insertions(+), 46 deletions(-)
--- verify: only MY commit is ahead of origin/master ---
5fa7f30 feat(storage): implement Google Drive backend (gdrive:// as first-class worker input)
```

70. assistant (2026-06-20T09:47:31.097Z)

Clean ? only my commit is ahead. Now push and open the PR:

71. user (2026-06-20T09:47:34.045Z)

```
remote:
remote: Create a pull request for 'feat/gdrive-storage-backend' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/gdrive-
storage-backend
remote:
branch 'feat/gdrive-storage-backend' set up to track 'origin/feat/gdrive-storage-backend'.
To https://github.com/avidullu/badminton-highlight-indexer.git
 * [new branch]      feat/gdrive-storage-backend -> feat/gdrive-storage-backend
```

72. user (2026-06-20T09:47:51.909Z)

https://github.com/avidullu/badminton-highlight-indexer/pull/258

73. assistant (2026-06-20T09:48:00.744Z)

PR opened: **[#258](https://github.com/avidullu/badminton-highlight-indexer/pull/258)**. Per the working-agreement (Claude-consumable current-state updated at every micro-checkpoint), let me record this checkpoint in memory.

74. user (2026-06-20T09:48:01.880Z)

```
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8       ---
9
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      > **Scope of this file = LIVE only:** the current handoff + *today's* checkpoints + open
PRs/decisions. Keep it read-first-short (?1 page). Older checkpoints are archived in [[current-
state-archive]] ? when today's checkpoints age out (next clean-slate day), move them there,
don't let them pile up here.
13
14      **? HANDOFF (2026-06-20) ? KHELSUTRA.GURU brand site: shipped, live, portable. Resume
the product threads below.** Separate track from the indexer rally-detection handoff that
follows. Full durable record + ramp-up: [[khelsutra-domain-launch]]. Repo =
**`avidullu/khelsutra`** (NOT this indexer repo).
15      - **DONE + LIVE this session (PRs #1?#8, all merged, repo clean):** owner bought
khelsutra.guru ? built & launched end-to-end. On **Cloudflare** (Worker `hidden-king-khelsutra`
+ D1 `khelsutra-access`; **git-connected ? merge `master` auto-deploys**; CI runs `npm test` =
20 green): landing page + `/capture` checklist + OG image + a **segmented Request-Access form**
(academy/individual + GPU + storage Qs + free text) ? persists to D1 **and emails a summary to
avi.dullu@gmail.com per registration** (Email Routing send binding ? VERIFIED live in inbox).
Valid SSL; `hello@` Email Routing verified.
16      - **PORTABLE + proven**: identical form runs on a dependency-free **Node + SQLite** host
(`site/server.js`), **live + browsable on the owner's DO droplet ? http://168.144.126.176:8787**
(systemd `khelsutra-site`, isolated `/srv/khelsutra-site`; SSH key `~/.ssh/khelsutra_droplet`).
Both-hosts compat ENFORCED in CI (`site/src/server.test.js` boots the real server). ? Droplet is
a SHARED storage testbed ? another session runs MinIO 9000/9001 + GCS emu 9023 +
`/root/do_gcs_*`; STAY CLEAR. Docs: `docs/REQUEST_ACCESS_FORM.md`, `docs/PORTABILITY.md`,
`NEXT_STEPS.md`.
17      - **? PENDING / next threads (owner ideas this session):** (1) **Marketing video** ? use
Grok/Gemini image-gen ? a short video themed for *serious enthusiasts* showing the whole
record?reel process ? embed on the site. (2) **Interactive per-rally selection** (big product
feature) ? the engine detects rallies ? present them in a UI so the user **picks which rallies
go into the highlight** (dynamic, NOT a fixed setting). ? **Design doc MERGED ?
[khelsutra#11](https://github.com/avidullu/khelsutra/pull/11); owner APPROVED ? decisions D8?D11
locked (status IN PROGRESS). ? CI regression-guards MERGED ?
[khelsutra#12](https://github.com/avidullu/khelsutra/pull/12): `site/src/deploy-config.test.js`
```

(pins wrangler.toml deploy invariants ? the #5 name-drift class), `site/src/assets.test.js` (static-site content contract), `.github/workflows/parity.yml` (scheduled+dispatch live-parity, NON-blocking); `npm test`?`node --test` discovery; suite 20?28 green; live-parity dispatched ? BOTH hosts byte-in-sync. NEXT = build MVP P1 (SPA scaffold in `web/`: Vite+React+TS+Tailwind + typed API client) ? P1?P5 on existing endpoints.** Built via a multi-agent design Workflow (research real engine contract ? 3 approaches ? synthesize) with **every load-bearing claim personally re-verified vs engine source**. Key finding: **MVP needs ZERO new engine endpoints** ? rides existing `POST /api/query` ? select `segment_ids` ? `POST /api/compile` ? `GET /api/highlights`. Recommended hybrid: selectable rally **grid** (workhorse) + **deterministic `parseQuery()`** (no LLM; length/count/order/shot_count ONLY) writing into ONE shared selection set + sticky compile footer. Honest scope: shot/player/score = roadmap (no column/detector); `server`/`receiver`/`events` = `motion`-only fabrications ? hidden; default `gemini` writes real-ish `shot_count`; per-rally *preview* blocked until a source/clip stream endpoint lands (deferred milestone = biggest UX-vs-effort lift). Owner decisions RESOLVED (D8?D11, 2026-06-20): (a) default selection **ON** (curate-by-removal); (b) preview-before-select **NOT** a ship-blocker (text-metadata MVP; preview = P9); (c) **job-tethered** MVP (no `GET /api/videos`); (d) `min_shot_count` = **warn, don't override** (override deferred ? Launch Report). Doc = `khelsutra/docs/PER_RALLY_SELECTION.md`. Pairs with the "ask for a reel" conversational builder already teased on the site. (3) productionize droplet (TLS + `vm.khelsutra.guru`), VM-path SMTP email (currently noop), Always-Use-HTTPS/HSTS, Web Analytics token, `/capture.html`?`/capture` footer link, delete any orphan `khelsutra-site` Worker, clean CF D1 test rows (`deploy-test@`/`email-test@`). Revoke the droplet SSH key when done. [[paper-coauthor-aim]] [[monetization-plan]]
18 - **? APPLIANCE DESIGN ? doc SHIPPED for review ? [khelsutra#13](https://github.com/avidullu/khelsutra/pull/13) (DRAFT, owner-gated; CI green) ? "locally-installed, UI-powered rally indexer + reel generator" (KhelSutra Studio).** Owner target (HEADLINE CUJ): a **hosted website** + a **NEW GPU-enabled machine (provider-agnostic: DO GPU droplet / GCE GPU VM / Cloud Run+GPU)** running the decoupled e2e; from the UI the user picks an input ? **Google Drive / bucket URI (S3/GCS/DO Spaces) / local upload?bucket** ? then **generates + watches + downloads** highlights **completely from the UI**. Framed as a **deployment PROFILE of the decoupled-compute arch** (engine `backend/storage/` StorageBackend + `python -m backend.worker {infer|train}` bucket pipeline; PRs #246?#254 on engine master, 936 green; **the CPU droplet 168.144.126.176 already hosts BOTH the site `/srv/khelsutra-site` AND the engine `~/rally`**, GPU MOCKED via stub). ? **OWNER PRIORITY (2026-06-20): MOVE TO A GPU-ENABLED MACHINE ASAP ? provider-agnostic: DO GPU droplet / GCE GPU VM / Cloud Run+GPU** (storage stays the S3/GCS bucket; GCS = storage, GCE/Cloud Run = compute ? do NOT conflate). Critical-path deps that are NOT mine to do unilaterally: (owner) stand up the GPU compute target + provide the cloud API token/creds ? DO runbook = engine `docs/SETUP_NEW_MACHINE.md` §2/§9 + `deploy/digitalocean/*`; **Cloud Run / GCE need the `backend.worker` CONTAINERIZED (likely a GAP; WASB's torch-1.11 conda env makes the image non-trivial) ? verify**. Cloud Run+GPU (scale-to-zero) is the strong fit for the per-video-billing economics (engine DECOUPLED_COMPUTE #246). (engine session) build **`NativeWasbRunner` (M3.5)** for real (non-mock) WASB. The **UI/appliance work runs in PARALLEL** (rides the existing FastAPI: chunked upload, `/api/process`?`/api/jobs`, `/api/query`, `/api/compile`, `/api/highlights`). ? Engine API has **NO auth (CORS `*`)** ? **EDGE AUTH required before any public exposure**. ? Google **Drive ingest may be a `gdrive` STUB** in `backend/storage/` (#249) ? verify before promising. Doc ? `khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md` (PR pending). The merged per-rally selection (#11) = ONE screen of this operator console. CUJ/live-test plan is a first-class doc section (owner asked re: live CUJ replay). **RECOMMENDATION (B+C hybrid):** reverse-proxied **UNIFIED ORIGIN** ? `web/` SPA behind edge auth, `/api/*` ? engine `127.0.0.1:8000` (CORS `*` moot; gate = tenancy boundary; bind verified `main.py:638`) ? driving the **EXISTING** FastAPI loop (upload?process?jobs?query?compile?highlights) with `storage=local`; **compute-as-a-setting**, bucket/remote-GPU worker = reserved scale path. **Verified-honesty refinements:** native WASB PARTIALLY wired (`fusion_hybrid` builds `NativeWasbRunner` :86; `trajectory_hybrid` refuses :57-63) + owner-byte-parity-gated; `ai_max_workers` default=**5** not 1; chunked-upload endpoints EXIST (`uploads.py:64/89/123`) but `app.js` never calls them (client-wiring only). **NET-NEW engine endpoints to COORDINATE** with the decoupled sessions: `/api/capabilities`, `/api/videos`, `/api/reels`, `/api/health`. **§10 owner decisions PENDING:** edge-auth (Caddy basicauth vs cloudflared+Access), retention defaults, per-video cost ceiling, GPU host choice, P5 bridge ownership. **Observability = first-class §5** (correlation-id e2e, structured logs per path, loud failures, health/metrics) per [[feedback-launch-quality-bar]]. Doc = `khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md`. **DELIVERY LOCKED (D10, 2026-06-20):** self-hosted **webserver + UI the user spins up** (NOT a desktop app; supersedes `DESKTOP_APP_PLAN.md`); user points at local storage+GPU/CPU OR cloud, UI drives the tool. **PROVISIONING ACCESS (2026-06-20):** `doctl` **INSTALLED v1.162.0** (winget

`DigitalOcean.Doctl`; at `%LOCALAPPDATA%\Microsoft\WinGet\Links\doctl.exe`, on PATH) ? **awaiting owner `doctl auth init`** (token entered interactively, NEVER pasted in chat ? cf. the already-rotated Spaces key; CLI authed on-box so the secret never enters the transcript) + a budget cap. Once authed: I verify GPU SKU/region/price via doctl (do NOT assume sgp1 has GPU) + set a DO billing alert at the cap, then propose-then-confirm the exact create cmd. **`gcloud` IS authed** here (avi.dullu@gmail.com, project `gen-lang-client-0306026826` ? likely needs GPU quota + billing verification before GCE/Cloud Run). Owner prefers **DO first** for GPU. Creating paid GPU infra = confirm-first every time; I propose exact cmd + hourly cost + teardown, owner OKs. [[monetization-plan]] [[khelsutra-domain-launch]]
19
20   ---
21
22   **? HANDOFF (2026-06-20) ? DECOUPLED_COMPUTE: M3.5 native WASB runner MERGED + DO?GCP pivot (GPU on GCP asia-south1). Resume from `deploy/gcp/README.md`.** Resumes the engine-side dependency the appliance block above was waiting on.
23   - ? **M3.5 MERGED ? [#256](https://github.com/avidullu/badminton-highlight-indexer/pull/256):** `NativeWasbRunner` (`backend/pipeline/detectors/native_wasb_runner.py`) runs `wasb_infer.py` NATIVELY on a Linux GPU box ? no WSL/conda-activate/mnt; `detector_impl="native"` seam; `device` cuda/mps/cpu (cuda default = byte-parity with WSL); weights/repo/python/scratch env-first (`WASB_WEIGHTS`/`WASB_REPO_DIR`/`WASB_PYTHON`/`WASB_SCRATCH`). `wasb_infer.py` got `--device` + device-keyed cache (back-compat). **`worker train --trajectory-source detector`** does REAL per-video extraction (fps PROBED via ffprobe + skip-on-miss, video-ext filter, graceful native-misconfig). Suite **998 green**, ruff+mypy clean, cov 83.7%. Adversarial multi-agent review run; fixes folded in. **Byte-parity (WSL CSV == native CSV) = owner GPU-box gate, NOT CI.**
24   - ? **DO?GCP PIVOT (ADR-010) + runbook ? [#257](https://github.com/avidullu/badminton-highlight-indexer/pull/257) (docs):** DO has NO Singapore/India GPU (NA-only droplets) ? GPU+bucket move to **GCP asia-south1 (Mumbai)**. `deploy/gcp/{README,provision.sh,teardown.sh}` = full runbook + live-resource inventory + switch-off. **PROVISIONED LIVE** (project `gen-lang-client-0306026826`, billing **Khelsutra Billing** `01420D-419EE0-53D83C`): all APIs on · bucket **`gs://khelsutra-rally-corpus`** (asia-south1) · SA **`rally-worker`** + key at **`~/.gcp/rally-worker-sa.json`** (local only, gitignored) · **`$100` budget `rally-100usd-guard`** (50/90/100%).
25   - ? **THE ONE GPU BLOCKER = `GPUS_ALL_REGIONS` quota = 0** (regional asia-south1 T4/L4 already 1). Owner: IAM&Admin?Quotas?"GPUs (all regions)"?1. **FAST PATH: a Spot T4 likely skips this wait** (preemptible gated by regional `PREEMPTIBLE_NVIDIA_T4_GPUS`=1, not the global cap) ? `deploy/gcp/README.md` §2/§3 spot variant. ? Bare Gemini project needed a one-time CONSOLE enable of Service Usage API (gcloud can't bootstrap it).
26   - **NEXT (owner-gated inputs):** owner files quota (or use Spot) + provides rotated Gemini key + WASB weights (`wasb_badminton_best.pth.tar`) + stages the 7 Done videos+labels into the bucket (Drive folder `1sHGL7iacnmM80ChT-p2cKy638gK504T_`, gdown by file-id) ? create VM (§3) ? port (reuse provider-agnostic `deploy/digitalocean/*`) ? **FIRST REAL `worker train --trajectory-source detector` + `worker infer` on held-out Video 1 GX010135_proxy.mp4.** Also: copy `/srv/khelsutra-site` off the DO droplet ? destroy droplet `579001481` ? rotate the exposed DO Spaces key. Upload options for inference = GCS URI / local / GDrive-via-gdown (`deploy/gcp/README.md` §5; `gdrive://` backend = tracked follow-up). [[monetization-plan]] [[working-agreement-merging]] [[paper-coauthor-aim]]
27
28   ---
29
30   **? HANDOFF (2026-06-19, clean-slate) ? NEXT SESSION START HERE for the prominent-court / multi-shuttle DESIGN journey.** Owner is "ramping up on the design mindset"; the prior session's context filled after a marathon (9 PRs merged today: #220/#237/#238/#239/#240/#241/#242/#243/#244). Owner asked for an honest call ? handed off here so the design work continues fresh.
31   - **WHERE WE ARE (prominent-court track, gap-closure G2):** designed + built + validated. `docs/PROMINENT_COURT_DETECTION.md` (tracked, C0-C8) is the design. `backend/pipeline/detectors/court_gate.py` (default-OFF, merged #243) has the **window-level** gate `gate_intervals_to_court` (drop a rally whose shuttle is MAJORITY outside a normalized `court_polygon`) ? wired into `rally_seg_eval.windows_for` via a per-video `court_polygon`. Validated on GX010142: cleanly drops the adjacent-court FPs incl the owner's rally #1, zero central false-drops. §4.5 (#244) captures the owner's **floor/white-line shuttle-floor-contact = deterministic rally-END** idea (C8, targets G3).
32   - **? VALIDATIONS LOGGED (2026-06-19, `scratch/validate_multishuttle_142.py`) ? answers the owner's 3 questions:**

33 - **Q1 CAN WASB GIVE ALL SHUTTLES? YES.** Code: `~/models/WASB-SBDT/src/detectors/postprocessor.py::_detect_blob_concomp` loops ALL connected components (no top-k/cap) ? detector returns every ?0.5 heatmap blob; the tracker is the sole single-shuttle reducer. Data: ?2-candidate frames are a STABLE venue property ? GX010142 31.2% / 143 32.3% / 145 26.7% / 147 27.7%, max 6/frame; the 2nd-best blob-mass (median 9.5-11.7) is comparable to the top (13.7-14.9) ? genuine, not noise. Bound: "all" = all the model fires ?0.5 on (a small/distant 2nd shuttle it misses won't appear; some ?2 are noise ? selection must disambiguate).
34 - **Q2 CAN WE PICK THE PROMINENT SHUTTLE? YES.** A court-AGNOSTIC greedy multi-target linker over the cache found 9 tracks in 21-25s; the **LONGEST/most-continuous (2.4s) is the ADJACENT one** (xmean-norm 0.90) ? naive "pick longest/most-confident" (? what the single tracker does) picks WRONG; a court+motion+vertical-excursion objective correctly flags the prominent fragments (xmean 0.44-0.68, in-court 100%). Caveats: prominent rally came back FRAGMENTED (~4 tracks ? need same-court fragment stitching; the time-windower mostly does this); separated cleanly because courts are X-apart HERE (X-overlapping courts need a reliable court signal ? Q3).
35 - **Q3 DO PERSON + FLOOR COALESCE? PARTLY ? fusion design sound; person input is USABLE (? EARLIER "BLIND DETECTOR" CLAIM RETRACTED).** ? **CORRECTION (2026-06-19):** my first Q3 read MISPARSED `output/person_GX010142.json` ? the 3rd field is box **AREA**, not confidence (dets are already filtered conf?0.5 in `person_detector.py::detect_and_track`). **FasterRCNN-ResNet50-FPN (BSD) IS firing**: **2.86 confident boxes/frame** (8551 over 2994 sampled frames; histogram 2-4/frame). During the prominent rally ~2.9 players/frame cluster centrally (cx 0.32-0.46) ? **CAN anchor the prominent court**. BUT players are present during rally#1 too (~2.2/frame, some central) ? **mere OCCUPANCY does NOT separate the FP from the real rally; the discriminator is player MOTION/activity (Lever A), not presence** (my "occupancy 0" was the area-as-conf bug, now fixed). Floor: prominent fragment descends only to y-norm 0.64 (no landing in 2.4s) + player-feet floor estimate unreliable ? **floor must come from COURT-LINE detection (§4.5/C8), not feet**. ? person+floor CAN coalesce; right person feature = **motion not presence**; no urgent detector replacement. Open (owner): person-signal QUALITY/court-assignment + the Qwen-unification strategy question (owned-model Gen-2, owner-gated) as pieces grow.
36 - **? GOLDEN TRUTH TEST ? GX010128 (2026-06-20, `scratch/eval_multitrack_gx128_golden.py`; fresh WASB run db84f3e6, resumable cache, keepawake+watchdog) ? VERDICT: multi-track does NOT beat baseline ? NO PR.** vs the 52 golden rallies: **baseline single-track F1 0.60 (P/R 0.49/0.77), tolF1 0.47, spurious 25.** Multitrack with the rally-likeness gate **REGRESSED to F1 0.34 (recall 0.33!)** ? the yspan/motion thresholds (tuned on 142's CLEARS) destroy recall; band widening barely helped (0.34?0.38) so the band is NOT the lever. Recall-preserving **court-ONLY selection recovers to F1 0.55 / recall 0.73 / tolF1 0.48 ? baseline ? but spurious 25=25 UNCHANGED.** ? **GX010128's over-fire is ON-COURT over-segmentation, NOT adjacent-court** (confirms the earlier state note) so the court-aware/multi-track lever has nothing to grab here. **Conclusions:** (1) multi-track is a **TARGETED lever** ? recovers specific tracker-discarded rallies (the 142 anchor) + drops adjacent-court FPs **where a separable adjacent court exists** (Boxhill_Doubles, where the merged #243 court_gate already showed ?F1 +0.078); it is **NOT a general F1 improver**. (2) selection MUST be **recall-preserving (court-membership only, never hand-tuned rally-likeness)**. (3) **Gemini-reference over-sold it** (precision-up/over-fire-halved looked good; golden truth = neutral-to-negative). Next real test = a **separable-adjacent-court golden clip** (Boxhill_Doubles) ? needs a GPU WASB run on its video (NOT local; GX010128 was the only local golden clip). **GHOST: confirmed on GX010128 ? fixed pixel (936,492) in 5617 frames (~20%!), but stationary ? dead-time, NOT a spurious-rally driver (spurious 25 with/without it); likely costs recall by occupying the tracker ? its own data-quality item, not the headline.** [[decision-rigor-norm]] [[eval-dual-set-regression]] [[paper-coauthor-aim]]
37 - **? (a) ZERO-GPU GENERALIZATION REPLAY (2026-06-19, `scratch/eval_multitrack_19june.py`) ? MIXED, NOT PR'd yet (honest):** multi-track court-aware selection vs single-track baseline, windowed identically (SERVED), scored vs the GEMINI REFERENCE (reference?truth, D3) on cached 142/143/145. Configured-band results: **142 prec 0.45?0.62 rec 0.83?0.75 ?; 143 prec 0.50?0.64 rec 0.95?0.90 ? (clean wins); 145 prec 0.31?0.39 rec 0.77?0.55 ?.** TOTAL: **precision 0.41?0.55, over-fire 2.08×?1.33× (nearly halved), recall 0.85?0.73 (real cost).** Diagnosis (visual `artifacts/multishuttle_142/replay_*.png`): the MECHANISM works; two gaps block a clean win ? (1) **court AUTO-derivation is fragile** (foreground-depth heuristic under-sized 145's band to 0.05-0.59, halving the court ? use configured polygon MVP meanwhile); (2) **rally-likeness thresholds (yspan/motion, hand-tuned on 142's high CLEARS) over-reject FLAT rallies on 145** ? needs DATA-DRIVEN tuning on golden, not hand-set thresholds (the n=1-overfit risk, now real). ? **HOLDING the PR** until the golden read disambiguates Gemini-reference noise. **GX010128 golden+ghost WASB run launched (bg, `output/19june_work/gx128_wasb.log`)** ? golden multicourt clip WITH the stuck-tracker ghost,

1080p/59.94 coords match golden labels ? the decisive truth test. Analysis staged: `scratch/eval_multitrack_gx128_golden.py`.
38 - **? SHUTTLE-ONLY MULTI-TRACK SELECTOR BUILT + VISUALLY VALIDATED (2026-06-19, `scratch/multitrack_select.py`; prototype, NO PR yet):** Layer-1 court-agnostic greedy multi-target linker over the cached candidates (zero GPU) + Layer-2 court-membership (configured prominent X-band, route-1 MVP) + rally-likeness (dur/yspan/motion) selector. Visually validated on 3 GX010142 windows (`artifacts/multishuttle_142/select_0|1|2.png`): GREEN=selected prominent rally, RED=rejected adjacent court+noise. **21-25s: 9 tracks?4 prominent selected; the NAIVE "pick-longest" track is the ADJACENT one (2.4s, xmean-norm 0.90) ? court-aware picks RIGHT where naive picks WRONG.** Adjacent-court contamination present in ALL 3 windows + consistently rejected (generalizes beyond the multi-shuttle case). ? Caveats seen: (1) **fragmentation** ? the prominent rally splits into several tracks (esp. long rallies); cleaner design = link?MERGE same-court fragments into a group?score the group (time-windower mostly stitches contiguous greens anyway); (2) court spec here = configured X-band, auto-derivation (shuttle-density or now-usable person-anchor) is the upgrade; (3) GX010142 only ? next = multi-clip replay + golden ?P/R/F1 before any flip. **Person session spun off (chip `task_01e00617`).**
39 - **? THE LIVE DESIGN CRUX (validated on real data ? pick this up):** the gate works on the single TRACKED shuttle, but the **tracker** (not the detector) collapses to one shuttle/frame. ? **CORRECTION (2026-06-19, `det_raw.jsonl` probe ? refutes the earlier "isn't in the data / can't be saved"):** WASB's **detector already emits MULTIPLE shuttle candidates/frame** (cached in `det_raw.jsonl`; ?2 candidates on **31%** of detected GX010142 frames). It is the single-target **online tracker** (`wasb_infer.py:537 tracker.update`) that discards all-but-one. MEASURED on GX010142 rally #3 (21-25s): the *tracked* shuttle is 94% adjacent-court (x-norm ~0.88), BUT the raw candidates ALSO carry a **prominent-court shuttle in 83% of the window's frames** (x-norm ~0.56), **both shuttles present in 76%**, and a greedy-NN reconstruction yields a **continuous prominent-court rally track** (75% coverage, 431px vertical excursion, 8.6px median per-frame jump, ~2.4s sustained). **So the prominent rally the owner saw IS in the data** ? recoverable by **multi-track selection over the already-cached candidates (zero GPU)**, just not by gate-tuning the single track. ? reframes the whole thread: the ceiling is **track SELECTION, not the eyes**. **NEXT DESIGN THREAD: multi-track + court-aware selection** (a window is valid if ANY sustained track lives in the prominent court) ? this is **recall-positive** (recovers the prominent rally, a G1 miss) AND precision-positive (drops the adjacent one), unlike the current subtractive single-track gate. It also subsumes the **ghost** (a stuck-pixel track has ~0 motion energy ? never selected). Repros: `scratch/analyze_multishuttle_142.py` + `scratch/reconstruct_prominent_track_142.py` + `scratch/overlay_tracks_142.py` (trajectory overlay) + `scratch/extract_multishuttle_frames_142.py`. ? **VISUALLY CONFIRMED (2026-06-19, `artifacts/multishuttle_142/tracks_overlay_crop.png`):** the green prominent track is a textbook **rally arc anchored to a player mid-stroke** (bottom-left) ? second player (center) ? a real foreground-court rally; the yellow tracker output is a tangle over the **far-right adjacent court** (player at right margin). Two shuttles, two courts, detector saw both. ? Corrected mid-investigation: the adjacent track is NOT a static fixture (528px y-span ? a real moving adjacent rally). ? Still n=1 clip; 31% multi-candidate includes detector noise (selection scoring must disambiguate). Owner collecting more two-shuttle examples; **DO NOT PR yet** (owner's call ? this is design).
40 - **OTHER OPEN THREADS:** (C6) **player-anchored polygon auto-derivation** ? person features ALREADY extracted on GX010142 ? `output/person_GX010142.json` (`scratch/extract_person_19june.py`; median 3 players/frame). Raw player COUNT doesn't separate rally #1 (need court-LOCALIZED occupancy via `fusion_features.in_court_counts` + the same polygon ? players inside the prominent court ? 0 during rally #1). Foot-points can auto-derive the polygon. (Analysis DONE) **multicourt quality workflow measured the gate vs golden on the 4 multicourt clips ? `output/multicourt_analysis.md`.** Honest result: the gate is a **REAL precision win ONLY where a genuinely separable adjacent court exists** ? **Boxhill_Doubles ?P +0.096 / ?F1 +0.078, recall FLAT, 20 adjacent-court FPs dropped (all median x>0.80)**. NULL/marginal elsewhere: mahadevpura_2 +0.025 (trims 4 edge FPs, no separable court), **GX010128 + Badminton_BXH_2 = exact NULL** (their over-fire is on-prominent-court over-merge/boundary FPs the court gate can't touch, NOT adjacent-court rallies). ? **2nd detection-layer finding (besides single-shuttle): STUCK-TRACKER artifacts** ? GX010128 has 284 pts at exactly x=1680px, BXH_2 ~41% of pts parked at court-center ? a fixed-pixel WASB false-lock (a data-quality bug, not a court issue). ? NO Gemini preds for these ? "closeness to Gemini" can't be direct; Gemini-on-multicourt run = a follow-up (alias degraded today). **Verdict: the prominent-court gate is a worthwhile precision lever for TRUE multicourt (separable adjacent court), not a cure-all for multicourt over-fire (which has 3 distinct causes: adjacent-court rallies [gate fixes], on-court over-merge [other levers], stuck-tracker [detection fix]).**
41 - **RAMP-UP KIT:** `docs/PROMINENT_COURT_DETECTION.md` (the design + tracker) ·

`backend/pipeline/detectors/court_gate.py` + `backend/pipeline/detectors/wasb_runner.py` (the single-shuttle constraint) · `tests/test_court_gate.py` · the litmus pattern in this session (windows_for + gate_intervals_to_court on a cached `output/wasb/<clip>_*_wasb.csv`) · `docs/RALLY_DETECTION_GAP_CLOSURE.md` (G1/G2/G3) · scratch: `analyze_19june.py`, `extract_person_19june.py`. **Branch:** currently `docs/court-floor-rally-end` (merged); base new work off `origin/master`. [[paper-coauthor-aim]] [[decision-rigor-norm]] [[working-agreement-merging]]
42
43   **Checkpoint:** 2026-06-19 (? **PROMINENT-COURT DESIGN+EXPERIMENT + DOC-STALENESS SWEEP ? 3 MORE PRs MERGED (#241/#242/#243)**). Owner directive: "extract person features" + "build up the 'prominent court' concept and make a design" (in multicourt, the foreground court covers the majority of the video area ? keep only shuttle rallies in it; may need a 'pseudo-3D' court model; experiment = `(x,y,visible)` ? `visible-in-prominent-court`) + "fix all staleness in the design docs and code, ultracode on, spin off agents" + **"auto accept, I'll be back, make and merge PRs"** (broad autonomy granted).
44   - ? **PROMINENT-COURT (gap-closure G2 / multicourt over-fire):** born from owner's **GX010142 rally #1** = a shuttle in the far-right adjacent court (verified: shuttle median X-norm **0.85**) that formed a 2.94s FP. **[#241](https://github.com/avidullu/badminton-highlight-indexer/pull/241) design doc** `docs/PROMINENT_COURT_DETECTION.md` (tracked, C0-C7; 2D floor polygon ? pseudo-3D play-volume; reuse `point_in_polygon`+`FusionConfig.court_polygon`+`PersonDetector.detections_per_frame`). **[#243](https://github.com/avidullu/badminton-highlight-indexer/pull/243) experiment** `backend/pipeline/detectors/court_gate.py` (default-OFF): **WINDOW-level** gate `gate_intervals_to_court` (drop a rally whose shuttle is MAJORITY out-of-court) wired into `rally_seg_eval.windows_for` via a per-video `court_polygon`. **KEY litmus finding:** point-level gating is TOO LEAKY ? rally #1 is 90% adjacent-court but a 10% central tail (12/121 pts) holds the window, surviving every 1D cut 0.66-0.82; the **window gate cleanly drops 6 rallies (all median X-norm 0.85-0.97 incl rally #1) with ZERO central false-drops.** ? confirms the design: 2D region + window-majority gate, NOT a 1D cut. ? Litmus polygon is crude/manual; **product path auto-derives it from PLAYER location (C4/C6)**. Eval n=15 UNCHANGED (no-op without polygon). 22 gate tests + suite 916 green, cov 83.07%.
45   - ? **PERSON FEATURES (C4) RUNNING (GPU):** `scratch/extract_person_19june.py` runs `PersonDetector.detections_per_frame` on GX010142 proxy ? `output/person_GX010142.json` for (a) player-anchored prominent-court derivation, (b) the rally-#1 "prominent players have not started" check (owner insight ? the PERSON-occupancy signal is the OTHER half of the rally-#1 fix; the court gate handles the shuttle side), (c) `shuttle_player_colocation` held-vs-flight. **shuttle_player_colocation IS BUILT** (`fusion_features.py:168`, default-OFF, unmeasured) ? corrected in the doc sweep.
46   - ? **DOC-STALENESS SWEEP [#242](https://github.com/avidullu/badminton-highlight-indexer/pull/242):** ran a **multi-agent Workflow** (7 themed auditors ? per-file adversarial verify; 30 candidates ? **25 confirmed**, each verified verbatim vs source). Fixed: signal build-status (Gate B / person cue / `shuttle_player_colocation` / `net_crossings` "[design]/NEW/unbuilt/eval-only" ? built-default-OFF-unmeasured), corpus n=6/9/12/13 ? **n=15**, CODE_MAP #233/#234 module moves, milestone-flip "owner-gated" ? LAUNCHED #204, CLAUDE.md/NEXT_STEPS GPU staleness, splits.py-DONE-#171, R2 `boundary_error_report` signature. ? One audit agent edited `R2_EVAL_METRIC_DESIGN.md` directly (rogue but accurate + in-scope) ? verified + included. Reusable: `output/_staleness_fixes.json`.
47   - ? **STILL IN-FLIGHT:** #243 CI (1 check, ? merge on green); person extraction (GPU). **NEXT:** analyze person features ? player-anchored polygon auto-derivation (C6) + rally-#1 player-activity check + colocation measurement; consider wiring the court gate into the SERVED fusion path. [[decision-rigor-norm]] [[working-agreement-merging]] [[paper-coauthor-aim]]
48
49   **Checkpoint:** 2026-06-19 (? **BOXHILL 19-JUNE LOCAL >7s EXTRACTION + GEMINI COMPARISON + 4 DUE-DILIGENCE PRs MERGED**). Owner: sync to remote head, verify sane, GPU local-only >7s extraction on 4-5 19-June videos ? new G: dir, Gemini comparison on 2-3 for golden-labeling, learnings on local-vs-Gemini gaps, fix issues via PRs as I go. **This box IS the owner's Windows machine** (CLAUDE.md "dev container lacks GPU/cv2/numpy" is fully STALE ? GPU+WSL+WASB all live here; RTX 2070 Super Max-Q).
50   - ? **Synced to origin/master** (local `master` is 50 behind + locked by stale `.claude/worktrees/tier1-windowing` worktree ? based work on `origin/master` directly). **Found + fixed a MASTER-BREAKING bug ? [#237](https://github.com/avidullu/badminton-highlight-indexer/pull/237) MERGED:** `test_ablation` called `ablation._is_fusion_schema` (moved to `experiment` by the #215/#235 SSOT refactor) ? **AttributeError at COLLECTION time ? whole local suite red** (green in CI only because the golden glob is empty there ? same green-CI/red-locally class as #236). Fix: import from the SSOT. Suite then 904 pass, cov 82.98%. **Quality NOT

regressed** (n=15 tIoU F1@0.5 0.468, R2 tolF1 0.353, identical).

75. assistant (2026-06-20T09:48:09.477Z)

The DECOUPLED_COMPUTE handoff (line 26) has a now-stale "`gdrive://` backend = tracked follow-up" note. Let me update it and add a brief checkpoint.

76. user (2026-06-20T09:48:19.715Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

77. assistant (2026-06-20T09:48:29.121Z)

All scope items are complete. Let me confirm CI kicked off on the PR:

78. user (2026-06-20T09:48:39.381Z)

Syntax sweep + import smoke (no GPU stack) pass 21s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819352
Golden fixtures cross-OS (windows-latest) pending 0 https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819363
Mypy (lenient baseline) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819330
Ruff lint pass 10s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819389
Full test suite (offline, mocked) pending 0 https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819338
Golden fixtures cross-OS (macos-latest) pass 15s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819360
Golden fixtures cross-OS (ubuntu-latest) pass 25s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819354

79. assistant (2026-06-20T09:48:47.894Z)

Lint, type, import-smoke, and golden cross-OS gates are all green. The full suite (offline) and Windows golden job are still running ? let me give them a bit and confirm the full suite, the key gate.

80. user (2026-06-20T09:48:48.345Z)

<tool_use_error>Blocked: sleep 75 followed by: gh pr checks 258 head -20. To wait for a condition, use Monitor with an until-loop (e.g. `until <check>; do sleep 2; done`). To wait for a command you started, use run_in_background: true. Do not chain shorter sleeps to work around this block.</tool_use_error>

81. user (2026-06-20T09:50:26.292Z)

=== final CI status ===
Full test suite (offline, mocked) pass 2m20s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819338
Golden fixtures cross-OS (macos-latest) pass 15s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819360
Golden fixtures cross-OS (ubuntu-latest) pass 25s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819354
Golden fixtures cross-OS (windows-latest) pass 49s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819363
Mypy (lenient baseline) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819330
Ruff lint pass 10s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819389
Syntax sweep + import smoke (no GPU stack) pass 21s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867466894/job/82473819352

82. assistant (2026-06-20T09:50:40.608Z)

All 7 CI checks green. Done.

Summary

Implemented the Google Drive storage backend so `gdrive://` is a first-class worker input. PR: **[#258](https://github.com/avidullu/badminton-highlight-indexer/pull/258)** (off `origin/master`, all CI green).

What changed

- **backend/storage/gdrive.py** ? `GDriveStorage` replaces the `NotImplementedError` stub, implemented against `pydrive2` behind the existing `storage-gdrive` extra. All six methods (`fetch_to_local` / `put` / `exists` / `stat` / `list` / `delete`) work; `signed_url` stays unsupported (callers fall back to fetch, per the base ABC).
  - **Id-addressed URI model** (Drive has no path namespace): `gdrive://<file-id>` = a file, `gdrive://<folder-id>` = list a folder's children, `gdrive://<folder-id>/<title>` = upload target. `list`/`put` return refs whose URI is `gdrive://<file-id>` ? the only reliably-fetchable address.
  - **Auth mirrors GCS**: service-account JSON named by the env (`GDRIVE_SERVICE_ACCOUNT_JSON`, falling back to the shared `GOOGLE_APPLICATION_CREDENTIALS`), **never from config**. The `pydrive2` import is **lazy**, so `import backend.storage` stays clean without the extra.
- The wire-through in `storage/__init__::_backend_class` + `parse_uri` was already routed (no change needed); refreshed the stale "follow-up" docstrings in `__init__.py`/`ref.py`.
- **tests/test_storage.py** ? added an in-memory fake Drive client (no network, mirroring the S3/GCS fakes) + roundtrip/overwrite-skip/missing-id/empty-id/signed-url tests; replaced the stale "not implemented" test; the lazy-import hygiene test now also guards `pydrive2`.
- **Docs**: `deploy/gcp/README.md` §5 and `docs/SETUP_NEW_MACHINE.md` §7c mark `gdrive://` implemented; `HANDOFF.md` / `config/models.py` / `pyproject.toml` notes refreshed.

Verification ? `run_all_tests.py`: **1003 passed**, coverage **83.74%** (?70 floor; `gdrive.py` at 87%, the 9 uncovered lines being the real `pydrive2`/`ServiceAuth` branch). `ruff` + `mypy backend training` clean. CI: all 7 jobs green.

Honest caveat (per the attribution working-agreement): the real `pydrive2` auth and round-trip against the owner's actual Drive are **not** verifiable here (no SA-JSON / network in this environment) ? the SDK path is exercised only via the injected fake client. On the box, the SA's email must be **shared on the target Drive file/folder** for reads to work.

I also recorded this as a checkpoint in your `current-state.md` memory (it closes the "Drive ingest may be a stub ? verify before promising" flag in the appliance handoff block).

83. user (2026-06-20T09:52:37.681Z)

Dont you see the https://drive.google.com/drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_ which actually maps to G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15

84. assistant (2026-06-20T09:53:29.039Z)

You're right, and that's an important correction to how I framed this. Let me verify the mount before I reason about what it means for the design.

85. user (2026-06-20T09:53:31.527Z)

```
=== Does the Drive-for-Desktop mount exist? ===
total 1
drwxr-xr-x 0 avidu 197609   0 Jun 18 11:00 .
drwxr-xr-x 0 avidu 197609   0 Jun 15 14:39 ..
-rw-r--r-- 0 avidu 197609   0 Jun 18 11:00 7 ? 1 ? 9.txt
drwxr-xr-x 0 avidu 197609   0 Jun 15 15:48 Video 1 - Badminton
drwxr-xr-x 0 avidu 197609   0 Jun 15 16:13 Video 11 - Pickleball
drwxr-xr-x 0 avidu 197609   0 Jun 15 16:44 Video 2 - Tabletennis
```

```
drwxr-xr-x 0 avidu 197609   0 Jun 15 17:04 Video 5 - Pickleball
drwxr-xr-x 0 avidu 197609   0 Jun 15 17:20 Video 7 - Badminton Multicourt
drwxr-xr-x 0 avidu 197609   0 Jun 15 17:33 Video 8 - Table tennis
drwxr-xr-x 0 avidu 197609   0 Jun 15 17:45 Video 9 - Badminton
drwxr-xr-x 0 avidu 197609   0 Jun 17 13:39 [Done] Video 10 - Badminton Multicourt
drwxr-xr-x 0 avidu 197609   0 Jun 18 11:50 [Done] Video 12 - Badminton Multicourt
drwxr-xr-x 0 avidu 197609   0 Jun 17 13:40 [Done] Video 13 - Badminton
drwxr-xr-x 0 avidu 197609   0 Jun 17 12:53 [Done] Video 14 - Badminton
drwxr-xr-x 0 avidu 197609   0 Jun 16 19:31 [Done] Video 3 - Badminton Multicourt
drwxr-xr-x 0 avidu 197609   0 Jun 18 12:16 [Done] Video 4 - Badminton
drwxr-xr-x 0 avidu 197609   0 Jun 17 13:15 [Done] Video 6 - Badminton
-rw-r--r-- 0 avidu 197609 246 Jun 19 17:00 desktop.ini

=== G: drive root ===
$RECYCLE.BIN
My Drive
Other computers
=== all drive letters ===
LICENSE.txt
ReleaseNotes.html
bin
cmd
dev
etc
git-bash.exe
git-cmd.exe
mingw64
proc
```

86. assistant (2026-06-20T09:53:41.009Z)

The mount is real ? that folder id maps exactly to the golden corpus (Video 1?14, `[Done]` markers matching the "7 Done" videos). Let me confirm a real video path inside and verify the storage layer already treats it as `local`.

87. user (2026-06-20T09:53:44.936Z)

```
=== a [Done] folder's contents (real video + labels?) ===
total 1517701
drwxr-xr-x 0 avidu 197609          0 Jun 18 12:16 .
drwxr-xr-x 0 avidu 197609          0 Jun 18 11:00 ..
-rw-r--r-- 0 avidu 197609 1554122102 Jun 15 16:54 GX020094_proxy.mp4
-rw-r--r-- 0 avidu 197609       1715 Jun 18 12:16 GX020094_proxy.rallies.csv
-rw-r--r-- 0 avidu 197609        246 Jun 19 17:00 desktop.ini
```

88. user (2026-06-20T09:53:45.990Z)

```
backend : local
bucket  : ''
key     : G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\[Done]
Video 4 - Badminton\clip.mp4
uri     : local://G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June
15\[Done] Video 4 - Badminton\clip.mp4
storage : LocalStorage
is Local: True
```

89. assistant (2026-06-20T09:55:08.669Z)

Confirmed. That folder id ? `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\` is a real **Google Drive for Desktop** mount ? it holds the actual corpus (`GX020094_proxy.mp4` 1.55 GB + `.rallies.csv`, the `[Done]` videos matching the "7 Done" in the handoff). And critically, a `G:\My Drive\...` path **already parses to the `local` backend ? `LocalStorage`** ? so it works today with zero new code.

So you're right, and I under-thought this. Let me state the honest re-evaluation plainly.

What I missed

For **your** workflow ? golden corpus, on this Windows box, local GPU/WSL ? the `gdrive://` API backend buys nothing. Drive-for-Desktop already exposes the whole folder tree as real files, and:

```bash
python -m backend.worker infer \
  --video-uri "G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\[Done] Video 4 - Badminton\GX020094_proxy.mp4" ...
```

?just works **today**. No service account, no sharing-with-an-SA-email, no pydrive2, no API quota. Drive streams + caches on access. It's strictly simpler and faster than what I built.

Where `gdrive://` (the API backend) actually earns its place

Only on a machine with **no mount**: the headless GCP GPU VM, a hosted appliance server, CI. And even there it's narrow, because:
- The GCP runbook already stages Drive?**GCS bucket** (co-located, reused across runs) ? the VM reads `gcs://`, not `gdrive://`.
- The service-account model I built can only see folders **explicitly shared with the SA's email**. True multi-tenant "point at *your* Drive" SaaS would need **OAuth user-consent**, not an SA ? a different thing.

So: `gdrive://` is a real but narrow tool (a headless server pulling from a Drive folder *you've shared with the SA*), and I oversold it as *the* Drive option without surfacing the obvious local-mount path first.

Let me get the exact current doc text so I can correct it honestly regardless of what we decide on the PR:

90. user (2026-06-20T09:55:09.481Z)

```
166     ### Upload options (how to get a video in + run inference)
167     The worker resolves `--video-uri` via the storage layer ? pick whichever fits:
168     - **GCS bucket (recommended, co-located):** `gcs://khelsutra-rally-corpus/videos/clip.mp4`. Upload with
169       `gcloud storage cp clip.mp4 gs://khelsutra-rally-corpus/videos/`.
170     - **S3 / DO Spaces / R2:** `s3://bucket/clip.mp4` (install `.[storage-s3]`, creds in `~/.aws/credentials`).
171     - **Local upload:** scp the file to the box and pass a bare path / `local:///root/clip.mp4` (identity passthrough, no copy).
172     - **Google Drive:** `gdrive://<file-id>` is a first-class input (`pip install -e .[storage-gdrive]`).
173       Auth via a **service-account JSON** named by the env ? `export GDRIVE_SERVICE_ACCOUNT_JSON=/path/sa.json`
174       (or reuse the shared `GOOGLE_APPLICATION_CREDENTIALS`); **share the Drive file/folder with the SA's
175       email** so it can read. IDs are opaque (no path namespace): `gdrive://<file-id>` for a file,
176       `gdrive://<folder-id>` to list a folder, `gdrive://<folder-id>/<title>` to upload. The legacy
177       `gdown <file-id> -O clip.mp4` ? local path still works for a quick one-off pull.
178
179     ---
180
181     ## 6. Switch off between sessions (cost control)
```

91. user (2026-06-20T09:55:25.684Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\gcp\README.md has been

updated successfully. (file state is current in your context ? no need to Read it back)

92. assistant (2026-06-20T09:55:28.537Z)

Now the same correction in `docs/SETUP_NEW_MACHINE.md` §7c ? lead with the mount:

93. user (2026-06-20T09:55:29.353Z)

```
212     ### 7c. Google Drive *(corpus source ? id-addressed)*
213
214     `pip install -e .[storage-gdrive]`. **Credentials** ? a service-account JSON; the SA's
email must be
215     granted access to the Drive file/folder (share it in the Drive UI). The path comes from
the env,
216     never config:
217     ```bash
218     export GDRIVE_SERVICE_ACCOUNT_JSON=/path/to/sa.json   # or reuse
GOOGLE_APPLICATION_CREDENTIALS
219     ```
220     Drive has no path namespace ? objects are addressed by an opaque **id**:
221     ```text
222     gdrive://<file-id>            # one file  (worker --video-uri, stat, delete)
223     gdrive://<folder-id>          # list a folder's immediate children
224     gdrive://<folder-id>/<title>  # upload target (put creates <title> in the folder)
225     ```
226     Same `worker infer` command with a `gdrive://<file-id>` `--video-uri`. *(Quick one-off
pull without
227     the backend: `gdown <file-id> -O clip.mp4`, then pass the local path.)*
228
229     **Bucket layout (for the bucket backends):** `videos/ labels/ manifest.json weights/
outputs/<video_id>/`
230     (see `docs/DECOUPLED_COMPUTE/04`).
231
232     ---
233
234     ## 8. Run training (Gen-0 rally head)
235
236     CPU-runnable, minutes ? but it needs a **labeled corpus** (videos + `.rallies.csv`
labels + a manifest),
237     which a fresh box doesn't have. Bring it via §7's bucket (`worker train`, pending) or
`scp` it up, then:
238     ```bash
239     python -m training.gen0.harness --manifest <manifest.json> --floor <baseline> --version
0.1.0
```

94. user (2026-06-20T09:55:48.999Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\SETUP_NEW_MACHINE.md has been
updated successfully. (file state is current in your context ? no need to Read it back)

95. assistant (2026-06-20T09:55:53.708Z)

Now let me commit these accuracy corrections to the branch (re-verifying HEAD first per
branching-safety):

96. user (2026-06-20T09:55:55.739Z)

```
branch: feat/gdrive-storage-backend
ahead of origin/master (should be ONLY my gdrive commit):
5fa7f30 feat(storage): implement Google Drive backend (gdrive:// as first-class worker input)
--- staging the two doc files ---
warning: in the working copy of 'deploy/gcp/README.md', LF will be replaced by CRLF the next
time Git touches it
M  deploy/gcp/README.md
```

```
M  docs/SETUP_NEW_MACHINE.md
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
```

97. user (2026-06-20T09:56:05.132Z)

```
[feat/gdrive-storage-backend 10abb64] docs(storage): lead the Drive option with the Drive-for-
Desktop local mount
 2 files changed, 33 insertions(+), 11 deletions(-)
To https://github.com/avidullu/badminton-highlight-indexer.git
   5fa7f30..10abb64  feat/gdrive-storage-backend -> feat/gdrive-storage-backend
```

98. assistant (2026-06-20T09:56:12.939Z)

Let me record the correction in the memory checkpoint, then I'll lay out the decision that's yours to make.

99. user (2026-06-20T09:56:14.510Z)

```
26      - **NEXT (owner-gated inputs):** owner files quota (or use Spot) + provides rotated
Gemini key + WASB weights (`wasb_badminton_best.pth.tar`) + stages the 7 Done videos+labels into
the bucket (Drive folder `1sHGL7iacnmM80ChT-p2cKy638gK504T_`, gdown by file-id) ? create VM (§3)
? port (reuse provider-agnostic `deploy/digitalocean/*`) ? **FIRST REAL `worker train
--trajectory-source detector` + `worker infer` on held-out Video 1 GX010135_proxy.mp4.** Also:
copy `/srv/khelsutra-site` off the DO droplet ? destroy droplet `579001481` ? rotate the exposed
DO Spaces key. Upload options for inference = GCS URI / local / **`gdrive://<file-id>`** (now
first-class, see checkpoint below) / GDrive-via-gdown (`deploy/gcp/README.md` §5).
[[monetization-plan]] [[working-agreement-merging]] [[paper-coauthor-aim]]
27
28      **Checkpoint:** 2026-06-20 (? **GDRIVE STORAGE BACKEND IMPLEMENTED ?
[#258](https://github.com/avidullu/badminton-highlight-indexer/pull/258), open, locally-
green).** Closes the `gdrive` STUB gap flagged in the appliance block ("Drive ingest may be a
stub ? verify before promising") + ADR-010/GCP runbook.
`backend/storage/gdrive.py::GDriveStorage` now real (pydrive2, `storage-gdrive` extra, lazy
import). **Id-addressed URI** (Drive has no path namespace): `gdrive://<file-id>` = a file
(worker `--video-uri`/stat/exists/delete), `gdrive://<folder-id>` = list children,
`gdrive://<folder-id>/<title>` = put. **Auth = service-account JSON from env**
(`GDRIVE_SERVICE_ACCOUNT_JSON` ? falls back to shared `GOOGLE_APPLICATION_CREDENTIALS`), never
config; SA email must be shared on the Drive file/folder. `signed_url` unsupported (callers fall
back to fetch). Tests mock an in-memory Drive (no network), mirroring the S3/GCS fakes; lazy-
import hygiene now guards pydrive2. Suite **1003 green**, cov 83.74%, ruff+mypy clean. ? NOT
locally verifiable: real pydrive2 auth/round-trip vs the owner's actual Drive (no SA-
JSON/network here) ? exercised only via the injected fake. Docs updated: `deploy/gcp/README.md`
§5, `SETUP_NEW_MACHINE.md` §7c, HANDOFF/config/pyproject. [[monetization-plan]] [[working-
agreement-merging]]
29
```

100. user (2026-06-20T09:56:27.145Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

101. assistant (2026-06-20T09:56:46.141Z)

I've verified the facts, corrected the docs to lead with the local mount, and logged the correction. The one thing left is genuinely your call ? and it changes what I do next.

102. user (2026-06-20T09:56:54.445Z)

The user did not answer the questions.

103. assistant (2026-06-20T09:57:14.051Z)

You didn't pick, so I'll take the **reversible** default: leave PR #258 **open and green** (docs corrected), and not merge or revert without your call ? both of those are harder to undo, and the work is done and passing. You can tell me "merge it" or "close it" whenever.

Let me confirm the docs-only commit kept CI green:

104. user (2026-06-20T09:58:42.563Z)

Full test suite (offline,: Full test suite (offline, mocked) pass 2m23s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867652451/job/82474279691
Golden fixtures cross-OS (macos-latest): Golden fixtures cross-OS (macos-latest) pass 17s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867652451/job/82474279636
Golden fixtures cross-OS (ubuntu-latest): Golden fixtures cross-OS (ubuntu-latest) pass 21s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867652451/job/82474279654
Golden fixtures cross-OS (windows-latest): Golden fixtures cross-OS (windows-latest) pass
1m12s https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27867652451/job/82474279649
Mypy (lenient baseline) pass: Mypy (lenient baseline) pass 33s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867652451/job/82474279632
Ruff lint pass 9s: Ruff lint pass 9s https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27867652451/job/82474279658
Syntax sweep + import: Syntax sweep + import smoke (no GPU stack) pass 19s
https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27867652451/job/82474279637
---summary---
7 checks passing

105. assistant (2026-06-20T09:58:57.931Z)

All 7 checks still green on the corrected branch. Here's where it stands.

Bottom line

You were right ? I jumped to the API backend and skipped the obvious path. **Verified on this box:**
- `?/drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_` ? `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\` (real Drive-for-Desktop mount, holds the actual corpus ? `GX020094_proxy.mp4` + `.rallies.csv`, all `Video N`/`[Done]` folders).
- A `G:\My Drive\?` path **already parses to the `local` backend** ? identity passthrough. So `--video-uri "G:\My Drive\?\GX020094_proxy.mp4"` works **today, zero new code**, no service account / sharing / quota.

`gdrive://` (the pydrive2 backend) only adds value on a **headless box with no mount** (GPU VM, hosted appliance) ? and even there it's narrow: the GCP runbook stages Drive?GCS anyway, and a service account only sees folders explicitly shared with it (true multi-tenant "point at *your* Drive" needs OAuth, not an SA).

What I did about it

- **Corrected the docs** (deploy/gcp/README.md §5 + docs/SETUP_NEW_MACHINE.md §7c) to **lead with the local mount** as the simplest Drive option and scope `gdrive://` to the headless case, with the SA-vs-OAuth caveat. Pushed as commit `10abb64`; CI green.
- Logged the correction (and the open decision) in memory.

Your call (I left it parked, not merged/closed)

[#258](https://github.com/avidullu/badminton-highlight-indexer/pull/258) is open and green. I asked which way to take it and you didn't pick, so I took the **reversible** default ? left it open, touched nothing destructive. Whenever you want:
- **"merge it"** ? keep the rescoped backend as headless-only insurance for the appliance path (my mild lean: it's done, green, lazy, costs nothing unused).
- **"close it"** ? I'll revert `gdrive.py` to the stub + drop the pydrive2 tests, and keep just the local-mount docs. Leanest option; your workflow is fully covered by local-mount + gdown + Drive?GCS.

If the immediate goal was just "use the golden corpus on this machine," you don't need #258 at all ? point the worker at the `G:\My Drive\?` path.

106. user (2026-06-20T09:58:59.650Z)

can you bake this in that user must mount the google drive and that is a fair assumption to be able to use google files locally

107. assistant (2026-06-20T10:01:20.468Z)

You've made the call: Google Drive = "mount it locally, use the file." Let me bake that in properly. The cleanest way that *also* keeps `gdrive://` working as a first-class input (the original ask) is to **reimplement `gdrive://` as a resolver over the local Drive-for-Desktop mount** ? no API, no service account, no pydrive2, pure-local and offline-testable. The backend will *require* the mount and assume it, exactly as you said.

Let me first check how the worker fetches the input (so I get the passthrough semantics right) and confirm the cross-platform mount paths.

108. user (2026-06-20T10:01:22.515Z)

```
89-    config = load_config(args.config) if args.config else load_config()
90-    _apply_overrides(config, args.set)
91:    storage_cfg = config.storage.model_dump()
92-
93-    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
--
96-
97-    # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download to
scratch.
98:    local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
99:    print(f"[worker] pulled {args.video_uri} -> {local_video}")
100-
101-    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be byte-
identical).
--
151-    for fn in sorted(os.listdir(out_local)):
152-        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
153:        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
154-        pushed.append(uri)
155-
--
244-    config = load_config(args.config) if args.config else load_config()
245-    _apply_overrides(config, args.set)
246:    storage_cfg = config.storage.model_dump()
247-
248-    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
--
254-
255-    corpus = args.corpus_uri.rstrip("/")
256:    label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
257-                        if u.lower().endswith(".csv"))
258-    if len(label_uris) < 2:
--
269-        if runner is None:
270-            return 2
271:        for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
272:            vname = storage.parse_uri(vuri).name
273-            if not vname.lower().endswith(_VIDEO_EXTS):
274-                continue  # skip sidecars (.srt/.json/thumbnails) so they can't shadow the
video
--
```

```
278-    specs: List[dict] = []
279-    for uri in label_uris:
280:        fname = storage.parse_uri(uri).name                    # e.g.
GX020094_proxy.rallies.csv
281-        name = fname.replace(".rallies.csv", "").replace(".csv", "")
282:        local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
283-        if args.trajectory_source == "detector":
284-            vuri = video_by_stem.get(name)
--
286-                print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
287-                continue
288:            local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
289-                                        config=storage_cfg)
290-            video_id = compute_video_id(local_video)
--
322-           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
323-    if args.floor:
324:        floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
325-                       if "://" in args.floor else args.floor)
326-        cmd += ["--floor", floor_local]
--
337-    for fn in sorted(os.listdir(bundle)):
338-        uri = f"{out_prefix}/weights/{args.version}/{fn}"
339:        storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
340-        pushed.append(uri)
341-    print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
```

109. assistant (2026-06-20T10:03:01.889Z)

The worker always fetches to a scratch dst (line 98) ? so `gdrive://` should behave exactly like the `local` backend, just rooted at the mount (copy out to scratch for a byte-stable MD5 input; passthrough when no dst). That means I can implement `GDriveStorage` as a thin resolver that delegates to `LocalStorage(base_dir=<mount>)`. Let me confirm the macOS mount path before I hardcode the auto-detect glob.

110. user (2026-06-20T10:03:08.446Z)

Web search results for query: "Google Drive for Desktop macOS mount path ~/Library/CloudStorage/GoogleDrive My Drive location"

Links: [{"title":"Google Cloud Storage mounted as a drive ? ExpanDrive | ExpanDrive","url":"https://www.expandrive.com/integrations/google-cloud-storage"},{"title":"Access to Google Drive Path - AppleScript | Mac OS X - MacScripter","url":"https://www.macscripter.net/t/access-to-google-drive-path/74300"},{"title":"[Solved] How to Mount Google Drive on Mac?","url":"https://iboysoft.com/howto/mount-google-drive-on-mac.html"},{"title":"Map Google Drive as network drive - How to mount on your Mac","url":"https://mac.eltima.com/mount-google-drive.html"},{"title":"How to mount cloud storage as a local drive (Simple steps)","url":"https://setapp.com/how-to/map-cloud-drives-on-mac"},{"title":"Macktez | Google Drive changes in macOS","url":"https://macktez.com/2023/google-drive-changes-in-macos/"},{"title":"How to Mount Google Cloud Storage as a Local Drive on macOS: A Practical Guide ? OurClone","url":"https://www.ourclone.app/blog/mount-google-cloud-storage-on-macos/"},{"title":"NUworkspace: Mounting a Cloud Drive","url":"https://services.northwestern.edu/TDClient/30/Portal/KB/PrintArticle?ID=1981"},{"title":"DEV Community","url":"https://dev.to/annawat/mount-google-drive-mac-easy-and-quickly-with-cloudmounter-4nk1"}]

Based on the search results, I found relevant information about Google Drive for Desktop's mount path on macOS:

Under the new architecture mandated by Apple, the full path goes through your Home folder (/Users/[yourname]/Library/CloudStorage/GoogleDrive/), which is different for every user.

This confirms that the full path goes through your Home folder
(/Users/[yourname]/Library/CloudStorage/GoogleDrive/) for Google Drive for Desktop on macOS.

So the location you mentioned?`~/Library/CloudStorage/GoogleDrive`?is correct for where Google
Drive for Desktop mounts on macOS. Your "My Drive" folder would be accessible at that path,
making the full path approximately `~/Library/CloudStorage/GoogleDrive/My Drive`.

REMINDER: You MUST include the sources above in your response to the user using markdown
hyperlinks.

111. assistant (2026-06-20T10:03:15.589Z)

Confirmed: macOS uses `~/Library/CloudStorage/GoogleDrive-<account>/My Drive`. I'll glob
`GoogleDrive*/My Drive` to cover both the account-suffixed and bare forms. Now let me
reimplement `GDriveStorage` as a mount-resolver.

112. assistant (2026-06-20T10:03:36.572Z)

```
"""Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.

We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google Drive
for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair requirement ?
it
needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on access. So
``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same code the
``local`` backend uses).

    gdrive://Sharing/clip.mp4   ->   <mount>/My Drive/Sharing/clip.mp4

Why an alias instead of just a raw path? Portability: the mount root differs per machine
(Windows ``G:\\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive``), so the
same
``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**, or set
it
explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a secret).

A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and use
``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend intentionally does
NOT
talk to the Drive API.
"""
from __future__ import annotations

import glob
import os
import string
from typing import Iterator, Optional

from backend.storage.base import StorageBackend
from backend.storage.local import LocalStorage
from backend.storage.ref import StorageRef, StorageStat


def _autodetect_mount_root() -> Optional[str]:
    """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google Drive for
    Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and Windows
    ``<letter>:\\My Drive`` (``G:`` is the install default). Linux/other ? configure
explicitly."""
    home = os.path.expanduser("~")
    for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage", "GoogleDrive*", "My
Drive"))):
        if os.path.isdir(p):
```

```
            return p
    if os.name == "nt":
        # G is the Drive-for-Desktop default; then scan the rest of the assignable letters.
        for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
            p = f"{letter}:\\My Drive"
            if os.path.isdir(p):
                return p
    return None


class GDriveStorage(StorageBackend):
    def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
        root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or _autodetect_mount_root()
        if not root or not os.path.isdir(root):
            raise RuntimeError(
                "Google Drive is not mounted. The gdrive:// backend reads files from a locally "
                "mounted Google Drive (install Google Drive for Desktop and sign in). Set "
                "GDRIVE_MOUNT_ROOT or storage.gdrive.mount_root to your 'My Drive' folder if it
"
                f"can't be auto-detected (tried: {root!r})."
            )
        self._root = os.path.abspath(root)
        self._local = LocalStorage(base_dir=self._root)  # all I/O is plain local-FS under the
mount

    def _local_ref(self, ref: StorageRef) -> StorageRef:
        """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path relative
to
        the mount root (``LocalStorage`` joins it under ``base_dir``)."""
        rel = ref.key.lstrip("/")
        key = os.path.join(*rel.split("/")) if rel else ""
        return StorageRef("local", "", key, f"local://{key}")

    def _to_gdrive_uri(self, abs_path: str) -> str:
        """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
        rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
        return f"gdrive://{rel}"

    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
        # The file is already local (via the mount): no dst => identity passthrough to the mount
        # path; a dst => copy out (a byte-stable scratch copy), exactly as the local backend
does.
        return self._local.fetch_to_local(self._local_ref(ref), dst, overwrite=overwrite)

    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
        self._local.put(local_path, self._local_ref(ref), content_type=content_type)
        return ref

    def exists(self, ref) -> bool:
        return self._local.exists(self._local_ref(ref))

    def stat(self, ref) -> StorageStat:
        return self._local.stat(self._local_ref(ref))

    def list(self, prefix) -> Iterator[StorageRef]:
        for r in self._local.list(self._local_ref(prefix)):
            uri = self._to_gdrive_uri(r.key)
            yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)

    def delete(self, ref) -> None:
        self._local.delete(self._local_ref(ref))
```

113. user (2026-06-20T10:03:36.613Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py has been
```

updated successfully. (file state is current in your context ? no need to Read it back)

114. assistant (2026-06-20T10:03:39.816Z)

Now update `parse_uri` so `gdrive://` is path-keyed (like `local`), and fix the docstring:

115. user (2026-06-20T10:03:42.068Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\ref.py has been
updated successfully. (file state is current in your context ? no need to Read it back)

116. assistant (2026-06-20T10:03:47.932Z)

Now `pyproject.toml` ? the backend is pure-stdlib now, so drop the pydrive2 extra:

117. user (2026-06-20T10:03:49.111Z)

```
67      # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68      # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69      # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70      # DO Spaces + MinIO via endpoint_url.)
71      storage-s3 = ["boto3>=1.34"]
72      storage-gcs = ["google-cloud-storage>=2.16"]
73      storage-gdrive = ["pydrive2>=1.19"]   # Google Drive (gdrive://) ? id-addressed,
service-account auth
74
```

118. user (2026-06-20T10:03:58.337Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\pyproject.toml has been updated
successfully. (file state is current in your context ? no need to Read it back)

119. assistant (2026-06-20T10:04:07.003Z)

Now the other docstrings that still reference the old pydrive2/id-addressed model:

120. user (2026-06-20T10:04:07.713Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local URI is actually used (install the ``storage-s3`` /
``storage-gcs``
5       / ``storage-gdrive`` extras). The default ``local`` backend needs no third-party deps
and
6       preserves today's behaviour byte-for-byte.
7
8       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
9       ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
10      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
11      """
```

121. user (2026-06-20T10:04:13.747Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage__init__.py has
been updated successfully. (file state is current in your context ? no need to Read it back)

122. user (2026-06-20T10:04:16.872Z)

```
1       """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
```

```
2
3       Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). Concrete backends
4       live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5       pulls ``boto3`` / ``google-cloud-storage`` / ``pydrive2``. See
``PLATFORM_ARCHITECTURE.md``
6       §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7       """
8       from __future__ import annotations
```

123. user (2026-06-20T10:04:23.169Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\base.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

124. user (2026-06-20T10:04:28.611Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

125. assistant (2026-06-20T10:04:33.878Z)

Now the tests. Let me read the current gdrive test section to replace it wholesale:

126. user (2026-06-20T10:04:34.597Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       Cloud backends are exercised with injected FAKE clients, so these tests need neither
4       boto3/google-cloud-storage/pydrive2 installed nor any network ? matching the repo's
offline ethos.
5       """
6       import subprocess
7       import sys
8
9       import pytest
10
11      from backend import storage
12      from backend.storage import StorageBackend, fetch, get_storage, list_uris, parse_uri,
put
13      from backend.storage.local import LocalStorage
14      from backend.storage.s3 import S3Storage
15      from backend.storage.gcs import GCSStorage
16      from backend.storage.gdrive import GDriveStorage
17
18
19      # ------------------------------------------------------------------ URI parsing
20
21      @pytest.mark.parametrize("uri,backend,bucket,key", [
22          ("/data/x.mp4", "local", "", "/data/x.mp4"),
23          ("rel/x.mp4", "local", "", "rel/x.mp4"),
24          ("C:/Users/a/x.mp4", "local", "", "C:/Users/a/x.mp4"),       # Windows path: no
'://'
25          ("C:\\Users\\a\\x.mp4", "local", "", "C:\\Users\\a\\x.mp4"),
26          ("local:///abs/x.mp4", "local", "", "/abs/x.mp4"),
27          ("s3://rally/videos/m.mp4", "s3", "rally", "videos/m.mp4"),
28          ("gs://rally-eu/w/model.json", "gcs", "rally-eu", "w/model.json"),   # gs:// -> gcs
29          ("gcs://rally/x", "gcs", "rally", "x"),
30          ("gdrive://FOLDERID/out/reel.mp4", "gdrive", "FOLDERID", "out/reel.mp4"),
31      ])
32      def test_parse_uri(uri, backend, bucket, key):
33          ref = parse_uri(uri)
34          assert (ref.backend, ref.bucket, ref.key) == (backend, bucket, key)
35
```

```
36
37      def test_parse_uri_rejects_unknown_scheme():
38          with pytest.raises(ValueError):
39              parse_uri("ftp://host/x")
40
```

127. user (2026-06-20T10:04:40.477Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

128. user (2026-06-20T10:04:44.923Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

129. assistant (2026-06-20T10:04:47.804Z)

Now replace the stale "not creds" test:

130. user (2026-06-20T10:04:53.341Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

131. assistant (2026-06-20T10:05:00.727Z)

Now replace the fake-Drive client + tests with mount-based tests. Let me read that block:

132. user (2026-06-20T10:05:01.172Z)

```
165     def test_s3_exists_false_on_missing():
166         assert S3Storage(client=_FakeS3Client()).exists(parse_uri("s3://b/none")) is False
167
168
169     # ---------------------------------------------------------------- GCS (fake client)
170
171     class _FakeBlob:
172         def __init__(self, store, bucket, name):
173             self.store, self.bucket, self.name = store, bucket, name
174             self.size = self.updated = self.etag = self.content_type = None
175
176         def upload_from_filename(self, src, content_type=None):
177             with open(src, "rb") as f:
178                 self.store[(self.bucket, self.name)] = f.read()
179             self.content_type = content_type
180
181         def download_to_filename(self, dst):
182             with open(dst, "wb") as f:
183                 f.write(self.store[(self.bucket, self.name)])
184
185         def exists(self):
186             return (self.bucket, self.name) in self.store
187
188         def reload(self):
189             self.size = len(self.store[(self.bucket, self.name)])
190
191         def delete(self):
192             self.store.pop((self.bucket, self.name), None)
193
194         def generate_signed_url(self, version, expiration, method):
195             return f"https://gcs.example/{self.bucket}/{self.name}?version={version}"
196
197
```

```
    class _FakeGCSClient:
        def __init__(self):
            self.store = {}

        def bucket(self, name):
            return type("_B", (), {"blob": lambda _self, key: _FakeBlob(self.store, name,
key)})()

        def list_blobs(self, bucket, prefix=""):
            return [_FakeBlob(self.store, bucket, k) for (b, k) in self.store
                    if b == bucket and k.startswith(prefix)]


    def test_gcs_backend_roundtrip_with_fake_client(tmp_path):
        be = GCSStorage(client=_FakeGCSClient())
        src = tmp_path / "w.json"
        src.write_text("{}")
        ref = parse_uri("gcs://rally/weights/w.json")
        be.put(str(src), ref)
        assert be.exists(ref)
        assert be.stat(ref).size == 2
        dst = tmp_path / "dl" / "w.json"
        be.fetch_to_local(ref, str(dst))
        assert dst.read_text() == "{}"
        assert [r.uri for r in be.list(parse_uri("gcs://rally/weights/"))] ==
["gcs://rally/weights/w.json"]
        assert be.signed_url(ref).startswith("https://gcs.example/rally/weights/w.json")
        be.delete(ref)
        assert not be.exists(ref)


    # ------------------------------------------------------------- Google Drive (fake
client)

    class _FakeDriveFile(dict):
        """Stand-in for pydrive2's GoogleDriveFile: a metadata dict bound to a shared
store."""

        def __init__(self, drive, metadata):
            super().__init__(metadata)
            self._drive = drive
            self._content = None  # staged by SetContentFile, flushed by Upload

        def GetContentFile(self, dst, **_):
            with open(dst, "wb") as fh:
                fh.write(self._drive.store[self["id"]]["content"])

        def FetchMetadata(self, **_):
            rec = self._drive.store.get(self.get("id"))
            if rec is None:
                raise KeyError(f"no such file: {self.get('id')!r}")  # mimics pydrive2
ApiRequestError
            self.update({k: v for k, v in rec.items() if k != "content"})

        def SetContentFile(self, path):
            with open(path, "rb") as fh:
                self._content = fh.read()

        def Upload(self, **_):
            new_id = self.get("id") or self._drive.next_id()
            content = self._content or b""
            self._drive.store[new_id] = {
                "id": new_id,
                "title": self.get("title", ""),
                "content": content,
```

```
                "parents": self.get("parents") or [],
                "mimeType": self.get("mimeType", "application/octet-stream"),
                "fileSize": str(len(content)),
                "modifiedDate": "2026-06-20T12:00:00.000Z",
                "md5Checksum": "deadbeef",
            }
            self["id"] = new_id

        def Delete(self):
            self._drive.store.pop(self.get("id"), None)


    class _FakeListResult:
        def __init__(self, drive, query):
            self._drive, self._query = drive, query

        def GetList(self):
            # Pull the folder-id out of "'<folder>' in parents and trashed=false".
            folder = self._query.split("'")[1] if "'" in self._query else ""
            return [_FakeDriveFile(self._drive, {k: v for k, v in rec.items() if k !=
"content"})
                    for rec in self._drive.store.values()
                    if any(p.get("id") == folder for p in rec.get("parents", []))]


    class _FakeDrive:
        """Minimal in-memory stand-in for pydrive2.drive.GoogleDrive (file-id keyed)."""

        def __init__(self):
            self.store = {}
            self._counter = 0

        def next_id(self):
            self._counter += 1
            return f"fid{self._counter}"

        def CreateFile(self, metadata=None):
            return _FakeDriveFile(self, dict(metadata or {}))
```

133. user (2026-06-20T10:05:05.150Z)

```
            return _FakeDriveFile(self, dict(metadata or {}))

        def ListFile(self, param=None):
            return _FakeListResult(self, (param or {}).get("q", ""))


    def test_gdrive_backend_roundtrip_with_fake_client(tmp_path):
        be = GDriveStorage(drive=_FakeDrive())
        assert isinstance(be, StorageBackend)
        src = tmp_path / "reel.mp4"
        src.write_bytes(b"VIDEO")
        # put into a folder -> returns a gdrive://<file-id> ref (the only address we can
fetch back).
        put_ref = be.put(str(src), parse_uri("gdrive://FOLDER/reel.mp4"),
content_type="video/mp4")
        assert put_ref.backend == "gdrive" and put_ref.uri.startswith("gdrive://")
        ref = parse_uri(put_ref.uri)
        assert be.exists(ref)
        st = be.stat(ref)
        assert st.size == 5 and st.content_type == "video/mp4" and st.etag == "deadbeef"
        assert st.modified is not None
        dst = tmp_path / "dl" / "reel.mp4"
        assert be.fetch_to_local(ref, str(dst)) == str(dst)
        assert dst.read_bytes() == b"VIDEO"
```

```
316         # list the parent folder -> the child, addressed by id, with its title surfaced as
`name`.
317         assert [r.uri for r in be.list(parse_uri("gdrive://FOLDER"))] == [put_ref.uri]
318         assert [r.name for r in be.list(parse_uri("gdrive://FOLDER"))] == ["reel.mp4"]
319         be.delete(ref)
320         assert not be.exists(ref)
321
322
323     def test_gdrive_fetch_skips_existing_without_overwrite(tmp_path):
324         drive = _FakeDrive()
325         be = GDriveStorage(drive=drive)
326         drive.store["abc"] = {"id": "abc", "content": b"NEW", "title": "x", "parents": []}
327         dst = tmp_path / "x.bin"
328         dst.write_bytes(b"OLD")
329         # Existing dst is left untouched unless overwrite=True (mirrors S3/GCS).
330         assert be.fetch_to_local(parse_uri("gdrive://abc"), str(dst)) == str(dst)
331         assert dst.read_bytes() == b"OLD"
332         assert be.fetch_to_local(parse_uri("gdrive://abc"), str(dst), overwrite=True) ==
str(dst)
333         assert dst.read_bytes() == b"NEW"
334
335
336     def test_gdrive_exists_false_on_missing():
337         assert GDriveStorage(drive=_FakeDrive()).exists(parse_uri("gdrive://nope")) is False
338
339
340     def test_gdrive_requires_an_id():
341         be = GDriveStorage(drive=_FakeDrive())
342         with pytest.raises(ValueError):
343             be.delete(parse_uri("gdrive://"))  # empty id
344
345
346     def test_gdrive_signed_url_not_supported():
347         # Drive has no presigned-URL story; callers must fall back to fetch_to_local.
348         with pytest.raises(NotImplementedError):
349             GDriveStorage(drive=_FakeDrive()).signed_url(parse_uri("gdrive://abc"))
350
351
352     # ------------------------------------------------------------- lazy-import hygiene
353
354     def test_importing_storage_does_not_pull_cloud_sdks():
355         # `import backend.storage` must not eagerly import boto3 / google-cloud-storage /
pydrive2,
356         # so the default install (and the CI lanes without those extras) stays clean.
357         code = (
358             "import sys, backend.storage; "
359             "assert 'boto3' not in sys.modules, 'boto3 imported eagerly'; "
360             "assert 'google.cloud.storage' not in sys.modules, 'gcs imported eagerly'; "
361             "assert 'pydrive2' not in sys.modules, 'pydrive2 imported eagerly'; "
362             "print('clean')"
363         )
364         r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True)
365         assert r.returncode == 0, r.stderr
366         assert "clean" in r.stdout
367
368
```

134. user (2026-06-20T10:05:41.962Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been updated successfully. (file state is current in your context ? no need to Read it back)

135. assistant (2026-06-20T10:05:46.761Z)

Now the docs need to match the mount model (they currently describe the API/service-account

version). Let me read and rewrite both sections:

136. user (2026-06-20T10:05:47.295Z)

```
166     ### Upload options (how to get a video in + run inference)
167     The worker resolves `--video-uri` via the storage layer ? pick whichever fits:
168     - **GCS bucket (recommended, co-located):** `gcs://khelsutra-rally-
corpus/videos/clip.mp4`. Upload with
169       `gcloud storage cp clip.mp4 gs://khelsutra-rally-corpus/videos/`.
170     - **S3 / DO Spaces / R2:** `s3://bucket/clip.mp4` (install `.[storage-s3]`, creds in
`~/.aws/credentials`).
171     - **Local upload:** scp the file to the box and pass a bare path /
`local:///root/clip.mp4` (identity passthrough, no copy).
172     - **Google Drive ? on a machine WITH Drive for Desktop (simplest, no new code):** the
mount exposes
173       Drive as a real filesystem path, so just pass that path ? the `local` backend handles
it (identity
174       passthrough, streamed/cached by Drive). On Windows it's `G:\My Drive\?`, on macOS
175       `~/Library/CloudStorage/GoogleDrive-<acct>/My Drive/?`. E.g. the golden corpus folder
176       `?/drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_` is
177       `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\`. **No
service account,
178       no API, no extra install.** ? The GPU VM is headless ? it has **no** mount; this
option is for the
179       owner's workstation (e.g. to copy Drive?bucket).
180     - **Google Drive ? on a HEADLESS box (no mount):** `gdrive://<file-id>` (`pip install -e
.[storage-gdrive]`).
181       Auth via a **service-account JSON** named by the env ? `export
GDRIVE_SERVICE_ACCOUNT_JSON=/path/sa.json`
182       (or reuse the shared `GOOGLE_APPLICATION_CREDENTIALS`); **the folder/file must be
shared with the SA's
183       email** (an SA sees nothing by default ? true multi-tenant "point at *your* Drive"
needs OAuth, not an SA).
184       IDs are opaque (no path namespace): `gdrive://<file-id>` for a file,
`gdrive://<folder-id>` to list,
185       `gdrive://<folder-id>/<title>` to upload. The simpler `gdown <file-id> -O clip.mp4` ?
local path also
186       works for a quick one-off pull. *(For the GPU run, prefer staging Drive?GCS once and
reading `gcs://`.)*
187
```

137. user (2026-06-20T10:06:00.788Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\gcp\README.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

138. assistant (2026-06-20T10:06:05.882Z)

Now `docs/SETUP_NEW_MACHINE.md` §7c:

139. user (2026-06-20T10:06:06.886Z)

```
212     ### 7c. Google Drive *(corpus source)*
213
214     **On a machine WITH Google Drive for Desktop (the simplest path ? no new code):** the
mount exposes
215     Drive as a **real filesystem path**, so just pass that path ? the default `local`
backend handles it
216     (identity passthrough; Drive streams + caches on access). Windows: `G:\My Drive\?`;
macOS:
217     `~/Library/CloudStorage/GoogleDrive-<acct>/My Drive/?`. The shared golden-corpus folder
218     `drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_` maps to
219     `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\`, so:
220     ```bash
```

```
221     python -m backend.worker infer --video-uri "G:\My Drive\?\[Done] Video 4 -
Badminton\GX020094_proxy.mp4" ?
222     ```
223     No service account, no API, no extra install. **A headless GPU VM has no mount** ? use
this on the
224     workstation (e.g. to copy Drive?bucket), and a bucket/`gdrive://` URI on the VM.
225
226     **On a HEADLESS box (no mount) ? the `gdrive://` API backend:** `pip install -e
.[storage-gdrive]`.
227     **Credentials** ? a service-account JSON; the SA's email must be granted access to the
file/folder
228     (share it in the Drive UI ? an SA sees nothing by default; true multi-tenant "point at
*your* Drive"
229     needs OAuth, not an SA). The path comes from the env, never config:
230     ```bash
231     export GDRIVE_SERVICE_ACCOUNT_JSON=/path/to/sa.json   # or reuse
GOOGLE_APPLICATION_CREDENTIALS
232     ```
233     Drive has no path namespace ? objects are addressed by an opaque **id**:
234     ```text
235     gdrive://<file-id>            # one file  (worker --video-uri, stat, delete)
236     gdrive://<folder-id>          # list a folder's immediate children
237     gdrive://<folder-id>/<title>  # upload target (put creates <title> in the folder)
238     ```
239     *(Quick one-off pull without the backend: `gdown <file-id> -O clip.mp4`, then pass the
local path.
240     For the GPU run, prefer staging Drive?GCS once and reading `gcs://`.)*
241
242     **Bucket layout (for the bucket backends):** `videos/ labels/ manifest.json weights/
outputs/<video_id>/`
243     (see `docs/DECOUPLED_COMPUTE/04`).
244
245     ---
246
247     ## 8. Run training (Gen-0 rally head)
248
249     CPU-runnable, minutes ? but it needs a **labeled corpus** (videos + `.rallies.csv`
labels + a manifest),
```

140. user (2026-06-20T10:06:24.613Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\SETUP_NEW_MACHINE.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

141. assistant (2026-06-20T10:06:28.515Z)

Now fix the HANDOFF.md lines I set last turn (they reference service-account auth):

142. user (2026-06-20T10:06:29.208Z)

```
22      - **Upload/run options** for inference: GCS bucket URI (recommended), local path/scp, or
23        `gdrive://<file-id>` (first-class, service-account auth ? `.[storage-gdrive]`). See
`deploy/gcp/README.md` §5.
```

143. user (2026-06-20T10:06:35.162Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\DECOUPLED_COMPUTE\HANDOFF.md
has been updated successfully. (file state is current in your context ? no need to Read it back)
```

144. assistant (2026-06-20T10:06:54.823Z)

Let me run the storage tests to validate the redesign before the full sweep:

145. user (2026-06-20T10:06:57.394Z)

```
Exit code 1
        the built-in open() function does.
        """
        if "b" not in mode:
            encoding = io.text_encoding(encoding)
>       return io.open(self, mode, buffering, encoding, errors, newline)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       FileNotFoundError: [Errno 2] No such file or directory:
'C:\\Users\\avidu\\AppData\\Local\\Temp\\pytest-of-
avidu\\pytest-635\\test_gdrive_put_writes_into_th0\\My Drive\\Sharing\\out.mp4'

..\..\AppData\Local\Programs\Python\Python313\Lib\pathlib\_local.py:537: FileNotFoundError
_____________________ test_gdrive_exists_false_on_missing _____________________

tmp_path = WindowsPath('C:/Users/avidu/AppData/Local/Temp/pytest-of-
avidu/pytest-635/test_gdrive_exists_false_on_mi0')

    def test_gdrive_exists_false_on_missing(tmp_path):
>       assert
GDriveStorage(mount_root=str(_mount(tmp_path))).exists(parse_uri("gdrive://nope.mp4")) is False
E       AssertionError: assert True is False
E        +  where True = exists(StorageRef(backend='gdrive', bucket='nope.mp4', key='',
uri='gdrive://nope.mp4'))
E        +    where exists = <backend.storage.gdrive.GDriveStorage object at
0x0000023ECB53AFD0>.exists
E        +      where <backend.storage.gdrive.GDriveStorage object at 0x0000023ECB53AFD0> =
GDriveStorage(mount_root='C:\\Users\\avidu\\AppData\\Local\\Temp\\pytest-of-
avidu\\pytest-635\\test_gdrive_exists_false_on_mi0\\My Drive')
E        +        where 'C:\\Users\\avidu\\AppData\\Local\\Temp\\pytest-of-
avidu\\pytest-635\\test_gdrive_exists_false_on_mi0\\My Drive' =
str(WindowsPath('C:/Users/avidu/AppData/Local/Temp/pytest-of-
avidu/pytest-635/test_gdrive_exists_false_on_mi0/My Drive'))
E        +          where WindowsPath('C:/Users/avidu/AppData/Local/Temp/pytest-of-
avidu/pytest-635/test_gdrive_exists_false_on_mi0/My Drive') =
_mount(WindowsPath('C:/Users/avidu/AppData/Local/Temp/pytest-of-
avidu/pytest-635/test_gdrive_exists_false_on_mi0'))
E        +    and   StorageRef(backend='gdrive', bucket='nope.mp4', key='',
uri='gdrive://nope.mp4') = parse_uri('gdrive://nope.mp4')

tests\test_storage.py:276: AssertionError
=========================== short test summary info ===========================
FAILED tests/test_storage.py::test_parse_uri[gdrive://Sharing/clip.mp4-gdrive--Sharing/clip.mp4]
FAILED tests/test_storage.py::test_gdrive_backend_roundtrip_over_local_mount
FAILED tests/test_storage.py::test_gdrive_put_writes_into_the_mount - FileNot...
FAILED tests/test_storage.py::test_gdrive_exists_false_on_missing - Assertion...
4 failed, 24 passed in 0.52s
```

146. assistant (2026-06-20T10:07:09.558Z)

The `parse_uri` failures reveal I edited the docstring but not the actual parsing logic. Let me fix the function:

147. user (2026-06-20T10:07:11.832Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\ref.py has been updated successfully. (file state is current in your context ? no need to Read it back)

148. user (2026-06-20T10:07:16.925Z)

```
............................                                             [100%]
28 passed in 0.45s
```

149. assistant (2026-06-20T10:07:25.423Z)

All 28 pass. Let me run ruff + mypy and sweep for any leftover references to the old API model:

150. user (2026-06-20T10:07:29.418Z)

```
All checks passed!
=== mypy ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 122 source files
```

151. user (2026-06-20T10:07:29.742Z)

```
=== leftover refs to API/pydrive/SA model (should be none in live docs/code) ===
backend/storage/gdrive.py:18:``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path.
This backend intentionally does NOT
backend/storage/ref.py:36:    """A backend + a location. ``bucket`` is the S3/GCS bucket or the
Drive folder-id;
deploy/gcp/README.md:181:  no mount** ? there, stage Drive?bucket and read `gcs://`, or `gdown
<file-id> -O clip.mp4` to a local path.
docs/DECOUPLED_COMPUTE/HANDOFF.md:61:- **Drive corpus** (source): folder
`1sHGL7iacnmM80ChT-p2cKy638gK504T_` (gdown-able by file-id). 7 `[Done]` = video+labels;
(end)
```

152. assistant (2026-06-20T10:07:37.896Z)

ruff + mypy clean. One stale docstring left ? `StorageRef` still says "bucket is the Drive folder-id." Let me fix it:

153. user (2026-06-20T10:07:39.007Z)

```
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket or the Drive folder-id;
37          for ``local`` it is empty and ``key`` carries the filesystem path."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
```

154. user (2026-06-20T10:07:47.472Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\ref.py has been updated successfully. (file state is current in your context ? no need to Read it back)

155. assistant (2026-06-20T10:07:51.175Z)

Now the full suite + the full-repo ruff (matching CI), to confirm nothing else regressed:

156. user (2026-06-20T10:09:36.149Z)

```
backend\worker\__init__.py                            0      0   100%
backend\worker\__main__.py                          246     31    87%
training\__init__.py                                  0      0   100%
training\gen0\__init__.py                             0      0   100%
```

```
training\gen0\harness.py                              108     11    90%
----------------------------------------------------------------------
TOTAL                                                8880   1448    84%
Required test coverage of 70% reached. Total coverage: 83.69%
1003 passed, 7 warnings in 100.05s (0:01:40)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

157. assistant (2026-06-20T10:09:56.186Z)

Full suite green (1003 passed, 83.69%). My new `_autodetect_mount_root` isn't exercised by the tests (they all pass an explicit root/env). Let me add a platform-robust test for it so the new code path is covered:

158. user (2026-06-20T10:10:01.591Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been updated successfully. (file state is current in your context ? no need to Read it back)

159. user (2026-06-20T10:10:08.498Z)

```
.............................                                        [100%]
=============================== tests coverage ================================
_____________ coverage: platform win32, python 3.13.13-final-0 _____________

Name                         Stmts   Miss  Cover   Missing
----------------------------------------------------------
backend\storage\gdrive.py       48      3    94%   39-40, 47
----------------------------------------------------------
TOTAL                           48      3    94%
29 passed in 0.55s
```

160. assistant (2026-06-20T10:10:31.457Z)

gdrive.py at 94% (the 3 missed lines are the macOS-mount-found branch, only reachable on a Mac, plus the `return None` covered on the Linux CI lane). Let me run the final CI-parity checks ? full suite + full-repo ruff:

161. user (2026-06-20T10:12:16.869Z)

```
Required test coverage of 70% reached. Total coverage: 83.77%
1004 passed, 7 warnings in 100.40s (0:01:40)
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

162. user (2026-06-20T10:12:17.950Z)

```
Exit code 1
artifacts\landing\make_og.py:23:1: E731 Do not assign a `lambda` expression, use a `def`
artifacts\landing\make_og.py:24:1: E731 Do not assign a `lambda` expression, use a `def`
Found 2 errors.
No fixes available (2 hidden fixes can be enabled with the `--unsafe-fixes` option).
```

163. assistant (2026-06-20T10:12:24.117Z)

Full suite **1004 passed**, **83.77%**; ruff clean (the only 2 errors are the untracked `artifacts/` file, absent in CI). Let me review the full diff, then commit to the same PR branch.

164. user (2026-06-20T10:12:25.919Z)

```
branch: feat/gdrive-storage-backend
ahead of origin/master:
10abb64 docs(storage): lead the Drive option with the Drive-for-Desktop local mount
5fa7f30 feat(storage): implement Google Drive backend (gdrive:// as first-class worker input)
=== working-tree changes (uncommitted) ===
warning: in the working copy of 'backend/storage/gdrive.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'deploy/gcp/README.md', LF will be replaced by CRLF the next
time Git touches it
 backend/config/models.py          |   4 +-
 backend/storage/__init__.py       |   7 +-
 backend/storage/base.py           |   6 +-
 backend/storage/gdrive.py         | 166 ++++++++++++++++--------------------
 backend/storage/ref.py            |  11 +--
 deploy/gcp/README.md              |  25 +++---
 docs/DECOUPLED_COMPUTE/HANDOFF.md |   3 +-
 docs/SETUP_NEW_MACHINE.md         |  42 +++++-----
 pyproject.toml                    |   4 +-
 tests/test_storage.py             | 178 ++++++++++++++++-------------------------
 10 files changed, 176 insertions(+), 270 deletions(-)
```

165. assistant (2026-06-20T10:12:32.577Z)

Net ?94 lines ? simpler than the API version. Let me stage the explicit paths and commit this pivot to the same PR branch:

166. user (2026-06-20T10:12:34.686Z)

```
warning: in the working copy of 'backend/storage/gdrive.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'deploy/gcp/README.md', LF will be replaced by CRLF the next
time Git touches it
--- staged (artifacts/ + KHELSUTRA doc stay untracked) ---
M  backend/config/models.py
M  backend/storage/__init__.py
M  backend/storage/base.py
M  backend/storage/gdrive.py
M  backend/storage/ref.py
M  deploy/gcp/README.md
M  docs/DECOUPLED_COMPUTE/HANDOFF.md
M  docs/SETUP_NEW_MACHINE.md
M  pyproject.toml
M  tests/test_storage.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
```

167. user (2026-06-20T10:12:50.866Z)

```
[feat/gdrive-storage-backend 0fed041] refactor(storage): make gdrive:// resolve a locally
MOUNTED Drive (no API/SDK)
 10 files changed, 176 insertions(+), 270 deletions(-)
--- ahead of origin/master (should be my 3 commits only) ---
0fed041 refactor(storage): make gdrive:// resolve a locally MOUNTED Drive (no API/SDK)
10abb64 docs(storage): lead the Drive option with the Drive-for-Desktop local mount
5fa7f30 feat(storage): implement Google Drive backend (gdrive:// as first-class worker input)
```

168. user (2026-06-20T10:13:00.692Z)

```
To https://github.com/avidullu/badminton-highlight-indexer.git
   10abb64..0fed041  feat/gdrive-storage-backend -> feat/gdrive-storage-backend
```

169. assistant (2026-06-20T10:13:03.028Z)

Now update the PR description to reflect the final mount-based design:

170. user (2026-06-20T10:13:24.460Z)

https://github.com/avidullu/badminton-highlight-indexer/pull/258

171. assistant (2026-06-20T10:13:28.686Z)

Let me update the memory checkpoint (the design decision is now resolved), then confirm CI:

172. user (2026-06-20T10:13:29.893Z)

28 **Checkpoint:** 2026-06-20 (? **GDRIVE STORAGE BACKEND IMPLEMENTED ? [#258](https://github.com/avidullu/badminton-highlight-indexer/pull/258), open, locally-green).** Closes the `gdrive` STUB gap flagged in the appliance block ("Drive ingest may be a stub ? verify before promising") + ADR-010/GCP runbook. `backend/storage/gdrive.py::GDriveStorage` now real (pydrive2, `storage-gdrive` extra, lazy import). **Id-addressed URI** (Drive has no path namespace): `gdrive://<file-id>` = a file (worker `--video-uri`/stat/exists/delete), `gdrive://<folder-id>` = list children, `gdrive://<folder-id>/<title>` = put. **Auth = service-account JSON from env** (`GDRIVE_SERVICE_ACCOUNT_JSON` ? falls back to shared `GOOGLE_APPLICATION_CREDENTIALS`), never config; SA email must be shared on the Drive file/folder. `signed_url` unsupported (callers fall back to fetch). Tests mock an in-memory Drive (no network), mirroring the S3/GCS fakes; lazy-import hygiene now guards pydrive2. Suite **1003 green**, cov 83.74%, ruff+mypy clean. ? NOT locally verifiable: real pydrive2 auth/round-trip vs the owner's actual Drive (no SA-JSON/network here) ? exercised only via the injected fake. Docs updated: `deploy/gcp/README.md` §5, `SETUP_NEW_MACHINE.md` §7c, HANDOFF/config/pyproject. **? OWNER CORRECTION (2026-06-20, commit `10abb64`): the `gdrive://` API backend is NOT needed for the owner's own corpus.** The Drive folder `1sHGL7iacnmM80ChT-p2cKy638gK504T_` is **mounted locally via Google Drive for Desktop** at `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\` (VERIFIED on this box: real videos + `.rallies.csv`, all 14 `Video N`/`[Done]` folders). A `G:\My Drive\?` path **already parses to the `local` backend** (`parse_uri` ? LocalStorage, identity passthrough) ? `--video-uri "G:\My Drive\?\GX020094_proxy.mp4"` works TODAY, zero new code, no SA/sharing/quota. So `gdrive://` only earns its place on a **headless box (GPU VM / appliance)** with no mount ? and even there the runbook stages Drive?GCS, and the SA model only sees SA-shared folders (multi-tenant needs OAuth). Docs now LEAD with the local mount; `gdrive://` scoped to headless. **DECISION PENDING (owner): keep the rescoped `gdrive://` backend (#258) for the headless niche, or close #258 and rely on local-mount + gdown + Drive?GCS staging.** [[monetization-plan]] [[working-agreement-merging]]
29

173. user (2026-06-20T10:13:41.083Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

174. assistant (2026-06-20T10:13:44.046Z)

Now wait for CI on the pivot commit:

175. user (2026-06-20T10:15:23.165Z)

=== final CI ===
Full test suite (offline, mocked) pass 2m8s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27868045115/job/82475296767
Golden fixtures cross-OS (macos-latest) pass 14s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27868045115/job/82475296759
Golden fixtures cross-OS (ubuntu-latest) pass 21s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27868045115/job/82475296751
Golden fixtures cross-OS (windows-latest) pass 1m5s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27868045115/job/82475296784
Mypy (lenient baseline) pass 27s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27868045115/job/82475296735
Ruff lint pass 7s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27868045115/job/82475296785

Syntax sweep + import smoke (no GPU stack) pass 20s https://github.com/avidullu/badminton-highlight-indexer/actions/runs/27868045115/job/82475296734
---
7 passing / 0 failing

176. assistant (2026-06-20T10:15:38.809Z)

All 7 CI checks green on the rewritten branch. Done.

Baked in: Google Drive = mounted Drive

I rewrote `gdrive://` to **resolve a locally mounted Google Drive** (Google Drive for Desktop) instead of calling the Drive API. The assumption you stated ? *to use Google files locally you mount the drive* ? is now the backend's contract: it requires the mount and fails fast with a clear "mount your Drive" message if it's absent. PR [#258](https://github.com/avidullu/badminton-highlight-indexer/pull/258) updated in place (3 commits show the API version ? this pivot), all CI green.

How it works ? `gdrive://<path>` is a portable alias for a path under your *My Drive*:
```
gdrive://Sharing/clip.mp4   ?   <mount>/My Drive/Sharing/clip.mp4   ?   plain local I/O
```
It delegates to `LocalStorage` rooted at the mount, so it behaves exactly like a local path (passthrough, or copy-to-scratch for the worker). The point of the alias over a raw `G:\My Drive\?` path is **portability** ? the mount root differs per machine, so the same URI resolves anywhere:
- **Auto-detected:** Windows `G:\My Drive`, macOS `~/Library/CloudStorage/GoogleDrive-<acct>/My Drive` ([confirmed path format](https://macktez.com/2023/google-drive-changes-in-macos/)).
- **Override:** `GDRIVE_MOUNT_ROOT` env or `storage.gdrive.mount_root` config (a path, not a secret).

What changed materially vs. the first version:
- **Dropped `pydrive2` entirely** ? no service account, no OAuth, no SDK, no `storage-gdrive` extra. `gdrive://` is pure-stdlib now, like `local`.
- `parse_uri` makes `gdrive://` path-keyed like `local://`.
- Tests rewritten against a `tmp_path` standing in for the mount (offline, no fakes).
- Docs (deploy/gcp/README.md §5, docs/SETUP_NEW_MACHINE.md §7c, HANDOFF, docstrings) all on the mount model.

The one place this *doesn't* apply is a **headless GPU VM / CI** ? no mount there, so you stage Drive?bucket and read `gcs://` (or `gdown`). The error message and docs say exactly that.

Verified: `run_all_tests.py` 1004 passed, coverage 83.77% (?70 floor), `gdrive.py` 94%; ruff + mypy clean; 7/7 CI checks green. Not exercised here: a real end-to-end `worker infer --video-uri gdrive://?` against `G:\My Drive` (storage resolution is tested; a full pipeline run is your GPU/real-video gate).

It's parked green ? say the word to merge.

Sources: [Google Drive changes in macOS ? Macktez](https://macktez.com/2023/google-drive-changes-in-macos/)

177. user (2026-06-20T10:15:40.716Z)

once you bake and test it, please merge the PR
yes and make sure to give a nice warning with a remedy to point to install and give the direction to include the folders to sync so that they can be pulled locally

178. assistant (2026-06-20T10:16:53.262Z)

On it. Before merging, I'll make the "not mounted" (and "mounted but not synced") errors genuinely helpful ? install link + the step to include the folder in syncing. Let me check the ruff line-length config so my multi-line messages stay clean, then rewrite the errors.

179. user (2026-06-20T10:16:53.604Z)

```
1       # Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would
2       # bury blame history for zero behavior value).
3       #
4       # Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle
5       # warnings (W) + bugbear (B). Deferred with intent: I (import-sort) and UP
6       # (pyupgrade) ? both are large mechanical diffs; adopt via a dedicated PR.
7       target-version = "py38"
8
9       # scratch/ = throwaway debugging scripts that may hit real APIs; it is also
10      # excluded from pytest collection (see pytest.ini).
11      extend-exclude = ["scratch"]
12
13      [lint]
14      extend-select = ["W", "B"]
15
16      [lint.per-file-ignores]
17      # Entrypoint scripts deliberately run setup (load_dotenv, logging config,
18      # sys.path) before importing backend modules ? import order is load-bearing.
19      "backend/main.py" = ["E402"]
20      "scripts/run_phase3_eval.py" = ["E402"]
21      "scripts/run_process_and_stitch.py" = ["E402"]
22
```

180. assistant (2026-06-20T10:17:00.772Z)

ruff doesn't enforce line-length (E501 not selected), so multi-line messages are fine. Now let me rewrite `gdrive.py` with the actionable errors and a "mounted but not synced" guard:

181. assistant (2026-06-20T10:17:26.662Z)

```
"""Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.

We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google Drive
for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair requirement ?
it
needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on access. So
``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same code the
``local`` backend uses).

    gdrive://Sharing/clip.mp4   ->   <mount>/My Drive/Sharing/clip.mp4

Why an alias instead of just a raw path? Portability: the mount root differs per machine
(Windows ``G:\\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive``), so the
same
``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**, or set
it
explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a secret).

A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and use
``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend intentionally does
NOT
talk to the Drive API. When the mount (or a file in it) is missing, errors point the user at the
fix: install Drive for Desktop and include the folder in syncing so it can be pulled locally.
"""
from __future__ import annotations

import glob
import os
import string
from typing import Iterator, Optional
```

```
from backend.storage.base import StorageBackend
from backend.storage.local import LocalStorage
from backend.storage.ref import StorageRef, StorageStat

_INSTALL_URL = "https://www.google.com/drive/download/"
```

Shared remedy line ? how to make a folder available locally. Drive for Desktop stream but a folder only appears once it's part of your synced Drive (and a *shared* folder mu be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but

```
_SYNC_REMEDY = (
    "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in Google "
    "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open Drive in
the "
    "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'. Drive
then "
    "downloads the bytes on first access."
)


def _not_mounted_msg(tried: Optional[str]) -> str:
    return (
        "Google Drive is not mounted, so the gdrive:// backend cannot resolve any path.\n"
        f"  Remedy:\n"
        f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
        f"  2. Then {_SYNC_REMEDY}\n"
        "  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive, macOS "
        "~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
        f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
    )


def _not_synced_msg(root: str, rel: str, uri: str) -> str:
    return (
        f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
        f"(expected at {os.path.join(root, *rel.split('/'))!r}).\n"
        f"  This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
        f"  (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
    )


def _autodetect_mount_root() -> Optional[str]:
    """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google Drive for
    Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and Windows
    ``<letter>:\\My Drive`` (``G:`` is the install default). Linux/other ? configure
explicitly."""
    home = os.path.expanduser("~")
    for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage", "GoogleDrive*", "My
Drive"))):
        if os.path.isdir(p):
            return p
    if os.name == "nt":
        # G is the Drive-for-Desktop default; then scan the rest of the assignable letters.
        for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
            p = f"{letter}:\\My Drive"
            if os.path.isdir(p):
                return p
    return None


class GDriveStorage(StorageBackend):
    def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
        root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or _autodetect_mount_root()
        if not root or not os.path.isdir(root):
```

```
            raise RuntimeError(_not_mounted_msg(root))
        self._root = os.path.abspath(root)
        self._local = LocalStorage(base_dir=self._root)  # all I/O is plain local-FS under the
mount

    def _local_ref(self, ref: StorageRef) -> StorageRef:
        """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path relative
to
        the mount root (``LocalStorage`` joins it under ``base_dir``)."""
        rel = ref.key.lstrip("/")
        key = os.path.join(*rel.split("/")) if rel else ""
        return StorageRef("local", "", key, f"local://{key}")

    def _require_present(self, ref: StorageRef) -> None:
        """Guard the read paths: a path missing from the mount almost always means the folder
        isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
downstream."""
        rel = ref.key.lstrip("/")
        abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
        if not os.path.exists(abs_path):
            raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))

    def _to_gdrive_uri(self, abs_path: str) -> str:
        """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
        rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
        return f"gdrive://{rel}"

    def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
        # The file is already local (via the mount): no dst => identity passthrough to the mount
        # path; a dst => copy out (a byte-stable scratch copy), exactly as the local backend
does.
        self._require_present(ref)
        return self._local.fetch_to_local(self._local_ref(ref), dst, overwrite=overwrite)

    def put(self, local_path, ref, *, content_type=None) -> StorageRef:
        self._local.put(local_path, self._local_ref(ref), content_type=content_type)
        return ref

    def exists(self, ref) -> bool:
        return self._local.exists(self._local_ref(ref))

    def stat(self, ref) -> StorageStat:
        self._require_present(ref)
        return self._local.stat(self._local_ref(ref))

    def list(self, prefix) -> Iterator[StorageRef]:
        for r in self._local.list(self._local_ref(prefix)):
            uri = self._to_gdrive_uri(r.key)
            yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)

    def delete(self, ref) -> None:
        self._local.delete(self._local_ref(ref))
```

182. user (2026-06-20T10:17:26.695Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\storage\gdrive.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

183. assistant (2026-06-20T10:17:30.608Z)

```
Now strengthen the tests ? assert the remedies are present, and add a "mounted but not synced"
case:
```

184. user (2026-06-20T10:17:40.590Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_storage.py has been updated successfully. (file state is current in your context ? no need to Read it back)

185. user (2026-06-20T10:17:50.769Z)

```
..............................                                           [100%]
30 passed in 0.44s
```

186. assistant (2026-06-20T10:17:56.279Z)

30 passed. Let me eyeball the actual rendered messages to confirm they read well:

187. user (2026-06-20T10:17:58.970Z)

```
================ NOT MOUNTED ================
Google Drive is not mounted, so the gdrive:// backend cannot resolve any path.
  Remedy:
  1. Install Google Drive for Desktop and sign in: https://www.google.com/drive/download/
  2. Then make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open Drive in
the browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'. Drive
then downloads the bytes on first access.
  3. If your 'My Drive' is not at the default location (Windows G:\My Drive, macOS
~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or
storage.gdrive.mount_root to point at it (auto-detect tried: None).

================ MOUNTED BUT NOT SYNCED ================
Google Drive is mounted ('G:\\My Drive') but 'gdrive://Sharing/Annotation Videos/[Done] Video 4
- Badminton/GX020094_proxy.mp4' is not present locally (expected at 'G:\\My
Drive\\Sharing\\Annotation Videos\\[Done] Video 4 - Badminton\\GX020094_proxy.mp4').
  This usually means the folder is not synced yet ? make sure the folder is INCLUDED in syncing
so its files can be pulled locally: in Google Drive for Desktop keep 'Stream files' on, and for
a folder shared with you, open Drive in the browser, right-click it -> 'Add shortcut to Drive'
so it appears under 'My Drive'. Drive then downloads the bytes on first access.
  (Install/repair Drive for Desktop if needed: https://www.google.com/drive/download/)
```

188. assistant (2026-06-20T10:18:05.274Z)

Both warnings read well and are actionable. Now the final verification gate (full suite + ruff + mypy), then I'll commit, push, and merge:

189. user (2026-06-20T10:19:55.237Z)

```
Required test coverage of 70% reached. Total coverage: 83.80%
1005 passed, 7 warnings in 102.81s (0:01:42)
============================================================================
 RESULT: PASS - all tests green
============================================================================
=== ruff (touched) ===
All checks passed!
=== mypy ===
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 122 source files
```

190. assistant (2026-06-20T10:20:00.907Z)

All green ? 1005 passed, 83.80%, ruff + mypy clean. Let me commit the warning improvements and push:

191. user (2026-06-20T10:20:02.158Z)

```
branch: feat/gdrive-storage-backend
0fed041 refactor(storage): make gdrive:// resolve a locally MOUNTED Drive (no API/SDK)
```

```
10abb64 docs(storage): lead the Drive option with the Drive-for-Desktop local mount
5fa7f30 feat(storage): implement Google Drive backend (gdrive:// as first-class worker input)
warning: in the working copy of 'backend/storage/gdrive.py', LF will be replaced by CRLF the
next time Git touches it
M  backend/storage/gdrive.py
M  tests/test_storage.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
```

192. user (2026-06-20T10:20:13.824Z)

```
[feat/gdrive-storage-backend 58a56e8] feat(storage): actionable gdrive:// errors ? install +
sync-the-folder remedy
 2 files changed, 64 insertions(+), 10 deletions(-)
To https://github.com/avidullu/badminton-highlight-indexer.git
   0fed041..58a56e8  feat/gdrive-storage-backend -> feat/gdrive-storage-backend
```

193. assistant (2026-06-20T10:20:16.419Z)

Now wait for CI to go green on the final commit, then merge:

194. user (2026-06-20T10:22:50.634Z)

```
=== CI on final commit ===
Full test suite (offline, mocked)  pass  2m20s  https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27868210771/job/82475706236
Golden fixtures cross-OS (macos-latest)  pass  16s  https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27868210771/job/82475706285
Golden fixtures cross-OS (ubuntu-latest)  pass  24s  https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27868210771/job/82475706264
Golden fixtures cross-OS (windows-latest)  pass  1m7s  https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27868210771/job/82475706259
Mypy (lenient baseline)  pass  30s  https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27868210771/job/82475706239
Ruff lint  pass  11s  https://github.com/avidullu/badminton-highlight-
indexer/actions/runs/27868210771/job/82475706249
Syntax sweep + import smoke (no GPU stack)  pass  19s  https://github.com/avidullu/badminton-
highlight-indexer/actions/runs/27868210771/job/82475706237
fail count: 0
```

195. assistant (2026-06-20T10:22:58.769Z)

All 7 CI checks green. Merging (squash, matching the repo's `(#NNN)` convention) and deleting the branch:

196. user (2026-06-20T10:23:04.002Z)

```
failed to run git: fatal: 'master' is already used by worktree at
'C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/tier1-windowing'

=== PR state ===
{"mergeCommit":"38c2d0f761c29d54b5d074cae28e0699ab94750b","mergedAt":"2026-06-
20T10:23:02Z","state":"MERGED"}
```

197. assistant (2026-06-20T10:23:13.132Z)

The PR **merged** (state MERGED, commit `38c2d0f`). The trailing git error was only the post-merge local checkout ? gh couldn't switch local `master` because a stale worktree (`.claude/worktrees/tier1-windowing`, a known sibling-session artifact) holds it. The merge on GitHub is complete. Let me confirm remote state and tidy the branch:

198. user (2026-06-20T10:23:19.354Z)

```
 - [deleted]         (none)     -> origin/refactor/tracknet-dedup-and-active-frames
```

```
 - [deleted]         (none)     -> origin/refactor/yolo-hybrid-process-video-helpers
   55ae408..38c2d0f  master     -> origin/master
=== master now contains the merge? ===
38c2d0f feat(storage): gdrive:// reads a locally MOUNTED Google Drive (no API) (#258)
=== gdrive on origin/master is the mount-resolver? ===
67:def _autodetect_mount_root() -> Optional[str]:
86:        root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or _autodetect_mount_root()
=== remote feature branch (should be gone) ===
```

199. assistant (2026-06-20T10:23:27.077Z)

Merge confirmed: `origin/master` ? `38c2d0f` ([#258](https://github.com/avidullu/badminton-highlight-indexer/pull/258)), the mount-resolver is on master, and the remote branch was auto-deleted. Let me mark it merged in memory:

200. user (2026-06-20T10:23:30.265Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

201. user (2026-06-20T10:23:43.707Z)

The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

202. assistant (2026-06-20T10:23:57.287Z)

Done ? baked, tested, and merged.

Merged: [#258](https://github.com/avidullu/badminton-highlight-indexer/pull/258) ? `master` (`38c2d0f`)

`gdrive://` ships as a mount-resolver over Google Drive for Desktop, with the failure modes turned into actionable warnings exactly as you asked.

The two warnings (install + sync-the-folder remedy):

Drive not mounted (`RuntimeError` at construction):

```
Google Drive is not mounted, so the gdrive:// backend cannot resolve any path.
  Remedy:
  1. Install Google Drive for Desktop and sign in: https://www.google.com/drive/download/
  2. Then make sure the folder is INCLUDED in syncing so its files can be pulled locally:
     in Google Drive for Desktop keep 'Stream files' on, and for a folder shared with you,
     open Drive in the browser, right-click it -> 'Add shortcut to Drive' so it appears
     under 'My Drive'. Drive then downloads the bytes on first access.
  3. If your 'My Drive' is not at the default location (Windows G:\My Drive, macOS
     ~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or
     storage.gdrive.mount_root to point at it.
```

Mounted but the file isn't synced yet (`FileNotFoundError` on fetch/stat) ? names the expected local path and points at the same "include the folder in syncing ? Add shortcut to Drive / keep Stream files on" remedy, instead of a bare "No such file."

The read paths (`fetch_to_local`, `stat`) guard presence, so the worker surfaces the remedy rather than a cryptic stack trace.

Verification (pre-merge gate, all reproduced locally):
- `python run_all_tests.py` ? **1005 passed**, coverage **83.80%** (?70 floor, up from baseline ? no regression). `gdrive.py` 94%.
- `ruff check` + `mypy backend training` ? clean.
- CI on the merge commit: **7/7 green** (full suite, mypy, ruff, import-smoke, golden cross-OS

×3).
- Tests assert both warning messages carry the install pointer + sync steps.

Two notes:
- The remote branch was auto-deleted on merge. Local `master` couldn't auto-checkout afterward because a stale worktree (`.claude/worktrees/tier1-windowing`, a sibling-session artifact) still holds `master` ? I left that untouched; your local checkout is still on `feat/gdrive-storage-backend`, now merged.
- Not exercised here: a real end-to-end `worker infer --video-uri gdrive://?` against the live `G:\My Drive` corpus ? the storage resolution and error paths are tested, but a full pipeline run on real video is your GPU gate.